

CI/CD avec Jenkins

v2.0.0 | 28/09/2026 | Auteur : Bauer Baptiste

Livre

Table des matières

1. Découvrir Jenkins	1
1.1. Introduction	1
1.2. Comment utiliser ce cours	1
1.2.1. Le parcours	1
1.2.2. Structure d'une étape	2
1.2.3. Les outils d'autonomie	2
1.3. Étape 0 : Comprendre l'intégration continue	3
1.4. Étape 1 : Installer Jenkins sur votre machine	4
1.5. Étape 2 : Premier job — vert, rouge, vert	9
1.6. Récapitulatif	12
2. Intégration continue de la calculatrice	13
2.1. Étape 3 : Relier un job au dépôt Git	13
2.2. Étape 4 : Construire et déployer avec Docker	16
2.3. Étape 5 : Tester, casser, réparer	19
2.3.1. Un job qui teste l'application	19
2.3.2. Casser le build	20
2.3.3. Réparer	21
2.4. Étape 6 : Pipeline et Jenkinsfile	22
2.4.1. 6.1 Enchaîner les jobs dans un pipeline	23
2.4.2. 6.2 Un Jenkinsfile de CI complet	23
2.4.3. 6.3 Exercice de débogage (approfondissement)	27
2.4.4. 6.4 Mesurer la couverture de tests (défi)	28
2.5. Récapitulatif	28
3. Préparer une instance AWS EC2	29
3.1. Étape 7 : Préparer un serveur AWS EC2	29
3.1.1. Création de la paire de clés SSH	29
3.1.2. Création de l'instance EC2	31
3.1.3. Installer et vérifier le client SSH	32
3.1.3.1. Vérifier si SSH est installé	32
3.1.3.2. Installer un client OpenSSH sur Windows	32
3.1.3.3. Installer un client OpenSSH sur Linux	32
3.1.4. Configurer les permissions de la clé privée	33
3.1.5. Tester la connexion depuis votre poste vers l'instance EC2	33
3.1.6. Installer Docker Engine sur l'instance	35
3.1.7. Vérifier l'étape	35
3.1.8. Arrêter l'instance en fin de séance	36
3.2. Récapitulatif	36
4. Jenkins sur AWS et webhook GitHub	37

4.1. Étape 8 : Jenkins sur AWS et webhook GitHub	37
4.1.1. 8.1 Lancer Jenkins sur le serveur	38
4.1.2. 8.2 Recréer le pipeline	39
4.1.3. 8.3 Autoriser GitHub à joindre Jenkins	39
4.1.4. 8.4 Créer le webhook	40
4.1.5. 8.5 Déclencher le pipeline par le webhook	41
4.1.6. 8.6 Faire vérifier le secret par Jenkins (approfondissement)	43
4.2. Récapitulatif	44
5. Dépôt privé et GitHub App	45
5.1. Étape 9 : Dépôt privé et GitHub App (approfondissement)	45
5.1.1. 9.1 Constater le problème	45
5.1.2. 9.2 Créer la GitHub App	46
5.1.3. 9.3 Installer l'App sur votre dépôt	48
5.1.4. 9.4 Générer et convertir la clé privée	49
5.1.5. 9.5 Enregistrer l'identifiant dans Jenkins	49
5.1.6. 9.6 Utiliser l'identifiant dans le pipeline	50
5.2. Pour aller plus loin	52
5.3. Récapitulatif	52
6. Mini-projet : votre propre chaîne CI/CD	53
6.1. Étape 10 : Mini-projet en autonomie (défi)	53
6.1.1. Choisir une application	53
6.1.2. Cahier des charges	53
6.1.3. Livrables	54
6.1.4. Grille d'auto-évaluation	54
6.1.5. Questions pour la soutenance	54
7. Mémo et glossaire	56
7.1. Mémo des commandes	56
7.1.1. Outils du TP	56
7.1.2. Jenkins (dans le dossier <code>jenkins/</code>)	56
7.1.3. Application (à la racine du dépôt)	56
7.1.4. Git	57
7.1.5. Syntaxe cron (scrutation)	57
7.1.6. Anatomie d'un Jenkinsfile déclaratif	57
7.2. Glossaire	58
7.3. Index des problèmes fréquents	59

1. Découvrir Jenkins

Version 2.0.0 | Dernière mise à jour : 28/09/2026

Durée de réalisation : 2 heures

Auteur : Bauer Baptiste

1.1. Introduction

Imaginez un monde où chaque modification de code est automatiquement testée, compilée et déployée sans que vous ayez besoin d'intervenir manuellement. C'est exactement ce que permet Jenkins. Jenkins est un outil open-source d'intégration continue et de livraison continue (CI/CD), utilisé par des millions de développeurs et d'équipes DevOps à travers le monde.

Sa force ? Automatiser vos tâches répétitives de build, de test et de déploiement, afin d'accélérer vos livraisons et de sécuriser vos projets logiciels. C'est un peu comme un chef d'orchestre qui coordonne tous les outils du développeur et de l'administrateur système.

1.2. Comment utiliser ce cours

Ce cours est conçu pour être suivi **en autonomie**. Il est découpé en **étapes** courtes : chacune n'introduit qu'une seule nouveauté et se termine par une vérification automatique.

1.2.1. Le parcours

Étape	Contenu	Chapitre	Niveau
0	Comprendre l'intégration continue	Découvrir Jenkins	★
1	Installer Jenkins sur votre machine	Découvrir Jenkins	★
2	Premier job : vert, rouge, vert	Découvrir Jenkins	★
3	Relier un job à un dépôt Git	Intégration continue de la calculatrice	★
4	Construire et déployer avec Docker	Intégration continue de la calculatrice	★
5	Tester, casser, réparer	Intégration continue de la calculatrice	★
6	Pipeline et Jenkinsfile	Intégration continue de la calculatrice	★ à ★★★★★
7	Préparer un serveur AWS EC2	Préparer une instance AWS EC2	★
8	Jenkins sur AWS et webhook GitHub	Jenkins sur AWS et webhook GitHub	★
9	Dépôt privé et GitHub App	Dépôt privé et GitHub App	★★
10	Mini-projet en autonomie	Mini-projet	★★★★

- ★ **Essentiel** : à faire obligatoirement, dans l'ordre.
- ★★ **Approfondissement** : pour aller plus loin une fois l'essentiel validé.
- ★★★ **Défi** : peu guidé, pour les plus rapides.

Les étapes 1 à 6 se font **sur votre propre machine**, sans frais. Le serveur AWS n'arrive qu'à l'étape 7, une fois que vous maîtrisez Jenkins.

1.2.2. Structure d'une étape

Chaque étape suit toujours le même plan :

Rubrique	Rôle
🎯 Objectif	Ce que vous saurez faire à la fin de l'étape.
💡 Comprendre	Les notions nécessaires, en quelques lignes.
🔧 À faire	Les manipulations. Chaque bloc de commandes indique où le lancer : 🖥️ votre poste, ☁️ le serveur EC2, 📦 le conteneur Jenkins.
✅ Vérifier	Ce que vous devez observer, puis la commande <code>./check.sh <étape></code> .
⊗ En cas de problème	Les erreurs les plus fréquentes et leur solution.
❓ Pour vérifier vos connaissances	Une ou deux questions de compréhension.

1.2.3. Les outils d'autonomie

`./check.sh` : la vérification automatique. À la racine de votre dépôt, lancez `./check.sh` suivi du numéro de l'étape (ex. `./check.sh 2`). Le script vérifie votre travail et, en cas d'erreur, indique quoi corriger :



```
— Étape 2 – Premier job
✓ Le job 'PremierJob' existe
✓ Vous avez provoqué un échec (build #2)
✗ Pas de build réussi après l'échec
  → Retirez 'exit 1' et relancez un build.
```

Ne passez à l'étape suivante que lorsque le script affiche **✓ Étape N validée !**.



Vous êtes bloqué ? Reprenez à partir de la correction. Le dépôt modèle contient une branche de correction pour les étapes importantes (`solution-etape-4`, `solution-etape-6`, `solution-etape-8`). Pour récupérer les fichiers d'une étape dans votre dépôt (les fichiers de la correction **remplacent** ceux de même nom ; vos autres fichiers sont conservés) :

```
git fetch https://github.com/bbauer02/jenkins-calculatrice.git
solution-etape-4
git checkout FETCH_HEAD -- .
git commit -m "Reprise à partir de la solution de l'étape 4"
git push
```

Un **mémo des commandes** et un **glossaire** sont disponibles dans le dernier chapitre.

1.3. Étape 0 : Comprendre l'intégration continue

🎯 **Objectif** : savoir expliquer ce que sont la CI et la CD, et ce que nous allons construire.

💡 Comprendre

Notion	Définition
Intégration continue (CI)	À chaque push , le code est automatiquement récupéré, construit et testé . Une erreur est détectée en quelques minutes, par la personne qui vient de l'introduire.
Livraison continue (CD)	Chaque version qui passe les tests produit un livrable (ici une image Docker) prêt à être déployé .
Déploiement continu	Le livrable validé est déployé automatiquement sur un serveur, sans intervention humaine.

Au fil de la séquence, nous allons construire la chaîne suivante pour une petite application web, une **calculatrice** écrite en Node.js :

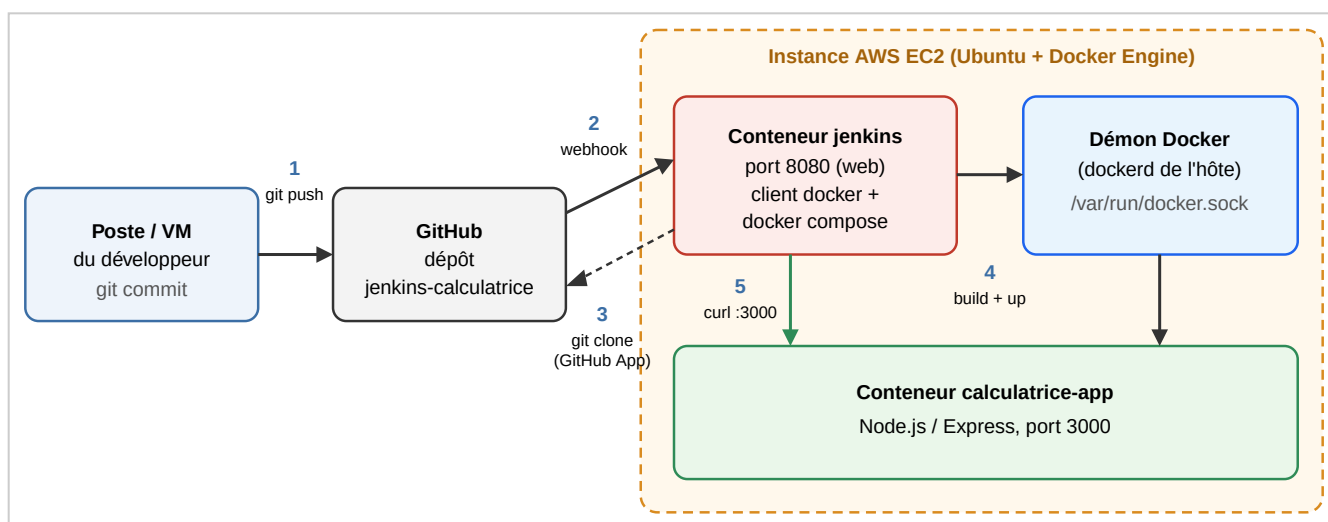


Figure 1. Architecture de la chaîne CI/CD (état final, à l'étape 8)

1. Le développeur pousse (**git push**) une modification sur GitHub.
2. GitHub prévient Jenkins grâce à un **webhook**.
3. Jenkins récupère le code (**git clone**).
4. Jenkins demande au **démon Docker** de construire l'image (tests compris) et de relancer le

conteneur de l'application.

5. Jenkins vérifie que l'application répond.

Pendant les étapes 1 à 6, tout tourne sur votre machine : Jenkins ira lui-même vérifier le dépôt régulièrement, au lieu d'être prévenu par GitHub.

Q1) Une équipe fusionne le travail de chacun une fois par mois, puis passe une semaine à corriger les conflits et les bugs. En quoi l'intégration continue changerait-elle leur façon de travailler ?

réponse 1 disponible.

1.4. Étape 1 : Installer Jenkins sur votre machine

🎯 **Objectif** : faire tourner Jenkins dans un conteneur Docker sur votre poste, et vous y connecter.

Prérequis

- **Docker** : Docker Desktop (Windows, macOS) ou Docker Engine (Linux), démarré.
- **Git** : sous Windows, installez [Git for Windows](#) et utilisez le terminal **Git Bash** pour toutes les commandes de ce cours.
- Un **compte GitHub**.

🔗 Comprendre

Jenkins est une application Java. Plutôt que de l'installer directement sur votre machine, nous allons le lancer dans un **conteneur Docker**, à partir de l'image officielle [jenkins/jenkins](https://hub.docker.com/r/jenkins/jenkins) (<https://hub.docker.com/r/jenkins/jenkins>).

[Explore](#) / [jenkins](#) / jenkins



jenkins/jenkins Sponsored OSS
By [Jenkins](#) · Updated 7 days ago
The leading open source automation server
[IMAGE](#)
[SECURITY](#) [INTEGRATION & DELIVERY](#) [MONITORING & OBSERVABILITY](#)
☆4.2K ↓1B+

[Overview](#) [Tags](#)

Jenkins Continuous Integration and Delivery server.

This is a fully functional Jenkins server, based on the weekly and LTS releases .



Jenkins

- To use the latest LTS: `docker pull jenkins/jenkins:lts-jdk17`
- To use the latest weekly: `docker pull jenkins/jenkins:jdk17`
- Lighter alpine, Windows and other JDKs based image also available
- (Recommended) Specific versions (to be pinned) are also available: `docker pull jenkins/jenkins:<version>--<jdk>` (Example: `jenkins/jenkins:2.414.3-jdk17` or `jenkins/jenkins:2.430-jdk21`)

Read [documentation](#) for usage

Tag summary

Recent tags
jdk21

Content type
Image

Digest
sha256:96203caba...

Size
275.5 MB

Last updated
7 days ago

[Run in Docker Desktop](#)

Requires Docker Desktop 4.37.1 or later.

Was this helpful? [👍](#) [👎](#)

Que signifie le tag `2.568.3-lts-jdk21` utilisé dans ce cours ?



- `2.568.3` : la version exacte de Jenkins. Elle est **figée** pour que l'interface corresponde aux captures du cours.
- `lts` (*Long-Term Support*) : version **stable**, recommandée en production. Une nouvelle ligne LTS est choisie environ toutes les 12 semaines, puis reçoit des correctifs toutes les 4 semaines. À l'inverse, la version *weekly* sort chaque semaine avec les dernières nouveautés.
- `jdk21` : la version de Java embarquée dans l'image. Jenkins exige Java 17 ou 21 : vous n'avez rien à installer.

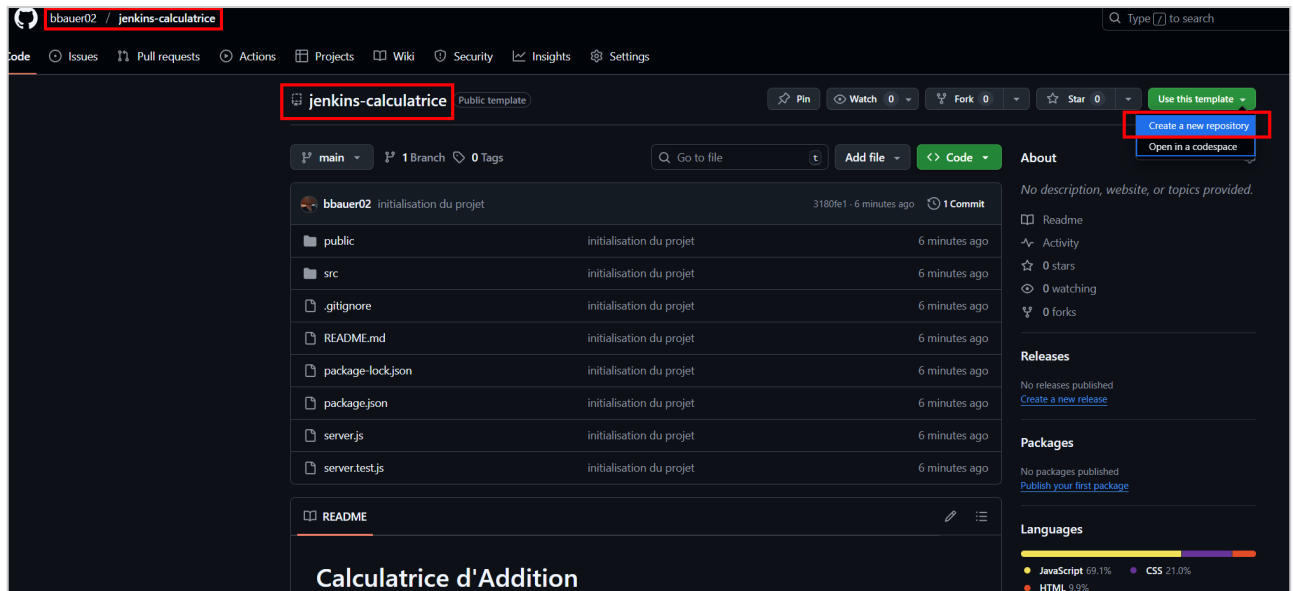
```
ubuntu@ip-172-31-38-30:~$ docker pull jenkins/jenkins:lts-jdk17
lts-jdk17: Pulling from jenkins/jenkins
f014853ae203: Pull complete
46de07924737: Pull complete
81c0d0b46f64: Pull complete
1958f49813c1: Pull complete
a8078a99224b: Pull complete
403fc70d0dc5: Pull complete
22c47ced1028: Pull complete
a82bddd92186: Pull complete
334e82eefb2d: Pull complete
e08bbdcaa5e8: Pull complete
d5232fba8477: Pull complete
2a7df0d1e300: Pull complete
Digest: sha256:be8c0f70f4cbab154cafa2f62b696147c12bfc2bf883eaa241c061bf5096df3b
Status: Downloaded newer image for jenkins/jenkins:lts-jdk17
docker.io/jenkins/jenkins:lts-jdk17
ubuntu@ip-172-31-38-30:~$
```

Le dépôt de l'application, que vous allez créer, contient un dossier `jenkins/` qui prépare tout :

Fichier	Rôle
<code>jenkins/Dockerfile</code>	Part de l'image officielle, ajoute le client Docker (pour que Jenkins puisse construire des images) et préinstalle les plugins listés dans <code>plugins.txt</code> .
<code>jenkins/casc/jenkins.yml</code>	Configuration de Jenkins <i>as code</i> : crée le compte administrateur, protège l'accès, met l'interface en français.
<code>jenkins/compose.yml</code>	Décrit comment lancer le conteneur (ports, volumes, accès à Docker).
<code>jenkins/init-env.sh</code>	Crée le fichier <code>jenkins/.env</code> : identifiants de l'administrateur, port, URL. Ce fichier contient un mot de passe : il n'est jamais versionné.

🔧 À faire

1. Rendez-vous sur le dépôt modèle <https://github.com/bbauer02/jenkins-calculatrice> et cliquez sur **Use this template** → **Create a new repository**. Nommez votre dépôt `jenkins-calculatrice` et laissez-le **public**.



2.  Configurez votre identité Git (une seule fois par machine) :

```
git config --global user.name "Prénom Nom"
git config --global user.email "<votre_email>"
git config --global init.defaultBranch main
```

3.  Clonez **votre** dépôt (remplacez **<votre_compte_github>**, sensible à la casse) :

```
git clone https://github.com/<votre_compte_github>/jenkins-calculatrice.git
cd jenkins-calculatrice
```



Lors du premier **git push**, Git vous demandera de vous authentifier auprès de GitHub. Sous Windows, une fenêtre de connexion GitHub s'ouvre automatiquement (*Git Credential Manager*).

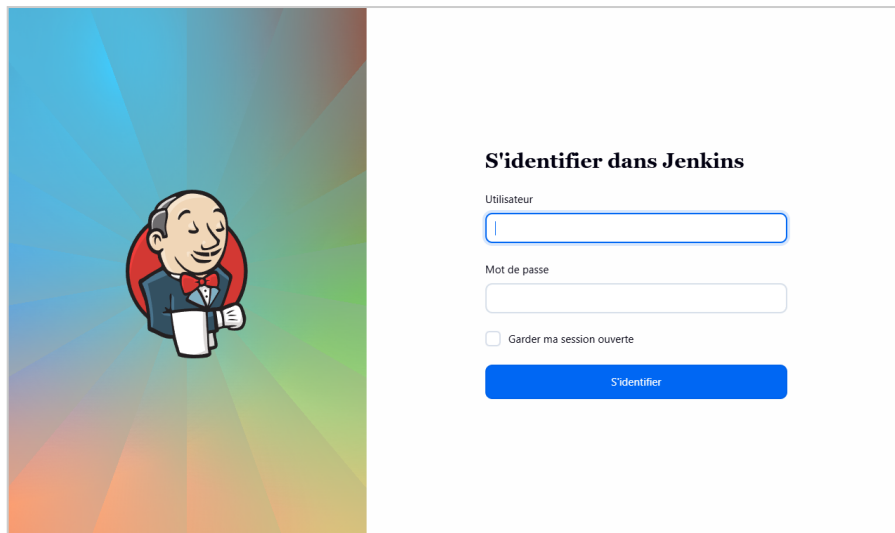
4.  Préparez le fichier **.env**, puis lancez Jenkins :

```
cd jenkins
./init-env.sh
docker compose up -d --build
```

Le script vous demande un identifiant, un **mot de passe robuste** (8 caractères minimum, sans apostrophe ; la saisie ne s'affiche pas), le port (laissez **8080** s'il est libre) et l'URL de Jenkins (laissez la valeur proposée). Si vous relancez le script plus tard, il vous demande d'abord s'il doit écraser le fichier **.env** existant : répondez **o**.

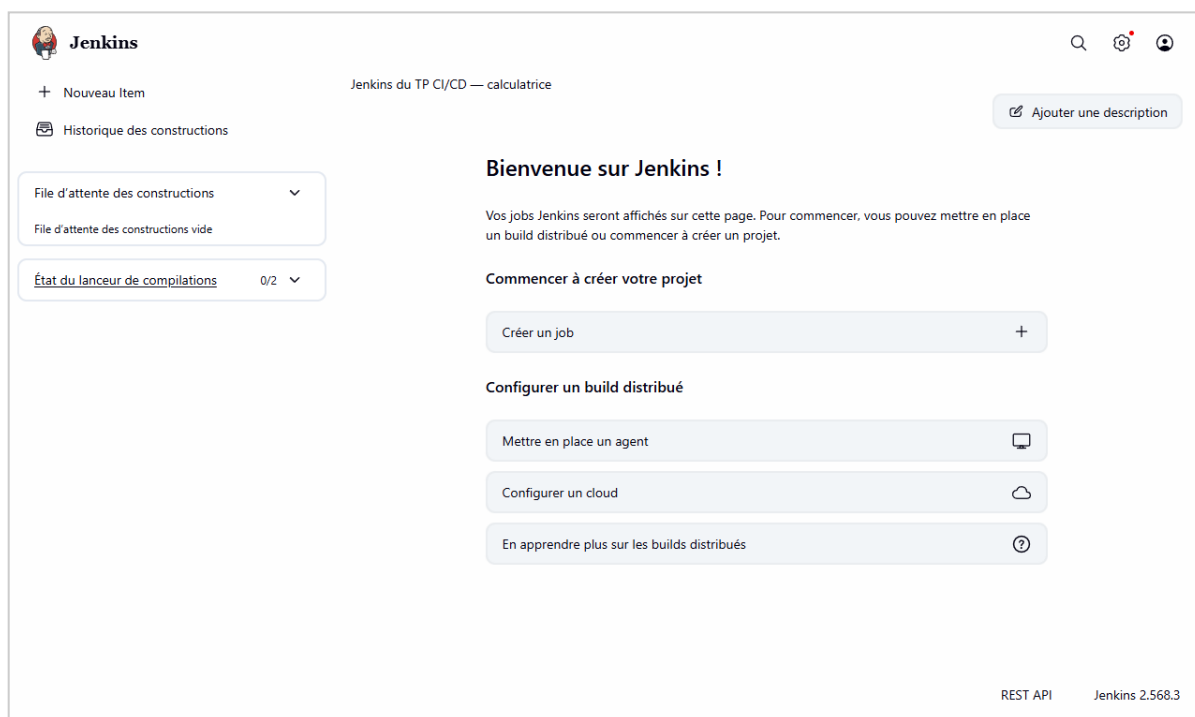
La première construction de l'image prend quelques minutes (téléchargement de Jenkins et des plugins).

5. Attendez que Jenkins démarre (environ une minute), puis ouvrez <http://localhost:8080> et connectez-vous avec l'identifiant et le mot de passe choisis.



✓ Vérifier

Vous arrivez sur le tableau de bord, avec le message « Bienvenue sur Jenkins ! ».



🖥 Depuis la racine du dépôt, lancez la vérification :

```
cd ..  
./check.sh 1
```

Table 1. Ce que fait `compose.yaml`

Option	Explication
<code>ports: "\${JENKINS_PORT:- 8080}:8080"</code>	Publie l'interface web de Jenkins sur le port choisi (8080 par défaut).

Option	Explication
<code>env_file: .env</code>	Transmet au conteneur les identifiants de l'administrateur, utilisés par <code>casc/jenkins.yaml</code> .
<code>JAVA_OPTS: -Djenkins.install.runSetupWizard=false</code>	Désactive l'assistant de premier démarrage : le compte administrateur est créé par la configuration <i>as code</i> .
<code>jenkins_home:/var/jenkins_home</code>	Volume persistant : jobs, historique des builds, identifiants. Supprimer ou recréer le conteneur ne les efface pas.
<code>/var/run/docker.sock:/var/run/docker.sock</code>	Donne à Jenkins l'accès au démon Docker de votre machine : les conteneurs que Jenkins lancera seront créés à côté de lui, et non à l'intérieur.
<code>group_add: \${DOCKER_GID}</code>	Ajoute l'utilisateur <code>jenkins</code> au groupe propriétaire du socket Docker (déecté par <code>init-env.sh</code>). Sans cela : <code>permission denied ... docker.sock</code> .
<code>extra_hosts: host.docker.internal:host-gateway</code>	Permet aux jobs de joindre votre machine (et donc l'application sur le port 3000) sous le nom <code>host.docker.internal</code> .



Monter `docker.sock` revient à donner les droits administrateur (root) sur la machine. Tout job capable d'exécuter `docker` peut prendre le contrôle de l'hôte. C'est acceptable pour un TP, mais en entreprise on utilise des **agents Jenkins dédiés**, Docker *rootless* ou des outils de build sans démon (Kaniko, Buildah). C'est aussi pour cela que l'accès à Jenkins est protégé par un mot de passe.

⊗ En cas de problème

Symptôme	Cause probable / solution
<code>./init-env.sh: Permission denied</code>	Lancez-le avec <code>bash init-env.sh</code> .
Docker ne répond pas	Démarrez Docker Desktop (ou <code>sudo systemctl start docker</code> sous Linux), puis recommencez.
<code>Bind for 0.0.0.0:8080 failed: port is already allocated</code>	Le port 8080 est déjà utilisé. Relancez <code>./init-env.sh</code> en choisissant un autre port (ex. <code>18080</code>), puis <code>docker compose up -d</code> . Jenkins est alors sur http://localhost:18080 : utilisez ce port partout où le cours indique <code>8080</code> .
<code>check.sh</code> : « Jenkins ne peut pas utiliser Docker » (<code>permission denied ... docker.sock</code>)	Le groupe du socket Docker n'est pas le bon. Relancez <code>./init-env.sh</code> (il le détecte automatiquement), puis <code>docker compose up -d</code> .
La page http://localhost:8080 ne répond pas	Jenkins démarre encore : suivez son démarrage avec <code>docker logs -f jenkins</code> (Ctrl+C pour quitter l'affichage). Attendez la ligne <code>Jenkins is fully up and running</code> .
Identifiant ou mot de passe refusé	Consultez <code>jenkins/.env</code> . Pour changer de mot de passe : relancez <code>./init-env.sh</code> , puis <code>docker compose up -d</code> .

Q2) Que deviennent les jobs et l'historique des builds si l'on supprime le conteneur avec `docker compose down`, puis qu'on le relance avec `docker compose up -d` ? Pourquoi ?

réponse 2 disponible.

1.5. Étape 2 : Premier job — vert, rouge, vert

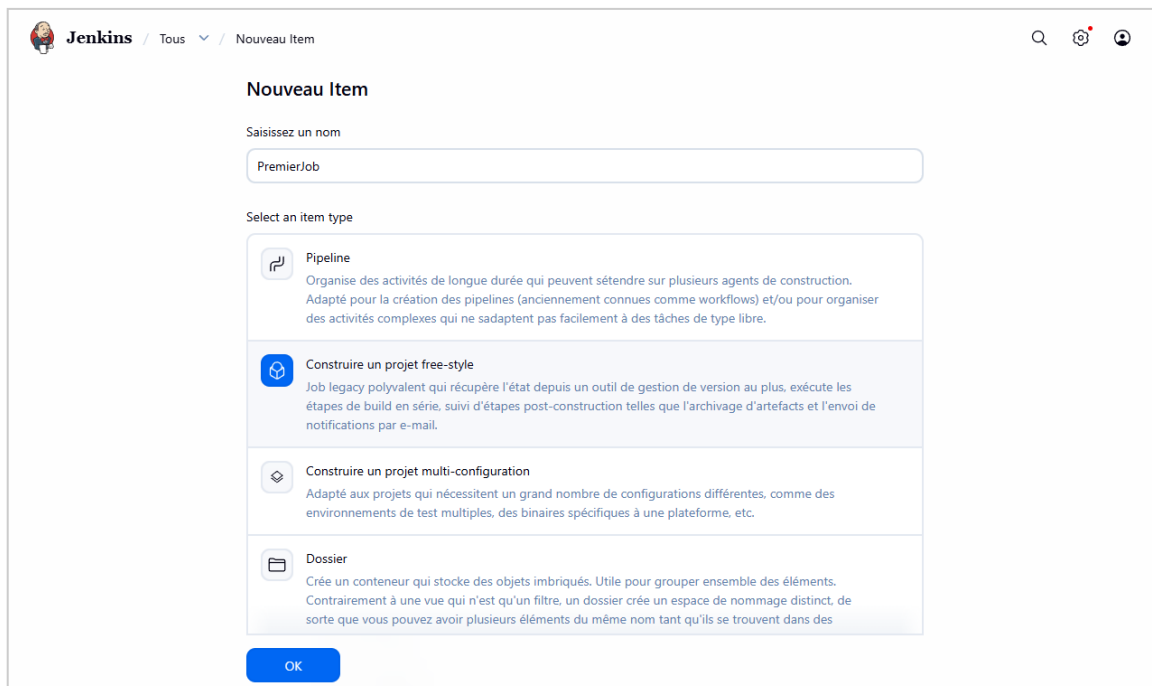
🎯 **Objectif** : créer un job, le lancer, lire sa sortie console, et comprendre ce qui fait qu'un build est réussi ou en échec.

💡 Comprendre

Notion	Définition
Job (ou projet)	Une tâche que Jenkins sait exécuter : une « recette ». Le type le plus simple est le projet free-style , configuré entièrement dans l'interface.
Build	Une exécution du job. Chaque build porte un numéro (#1 , #2 ...) et garde sa propre sortie console.
Workspace	Le dossier de travail du job, dans le conteneur : <code>/var/jenkins_home/workspace/<nom du job></code> .
Statut	✅ Succès si toutes les commandes se terminent avec le code de retour <code>0</code> . ❌ Échec dès qu'une commande renvoie un code différent de <code>0</code> .

🔧 À faire

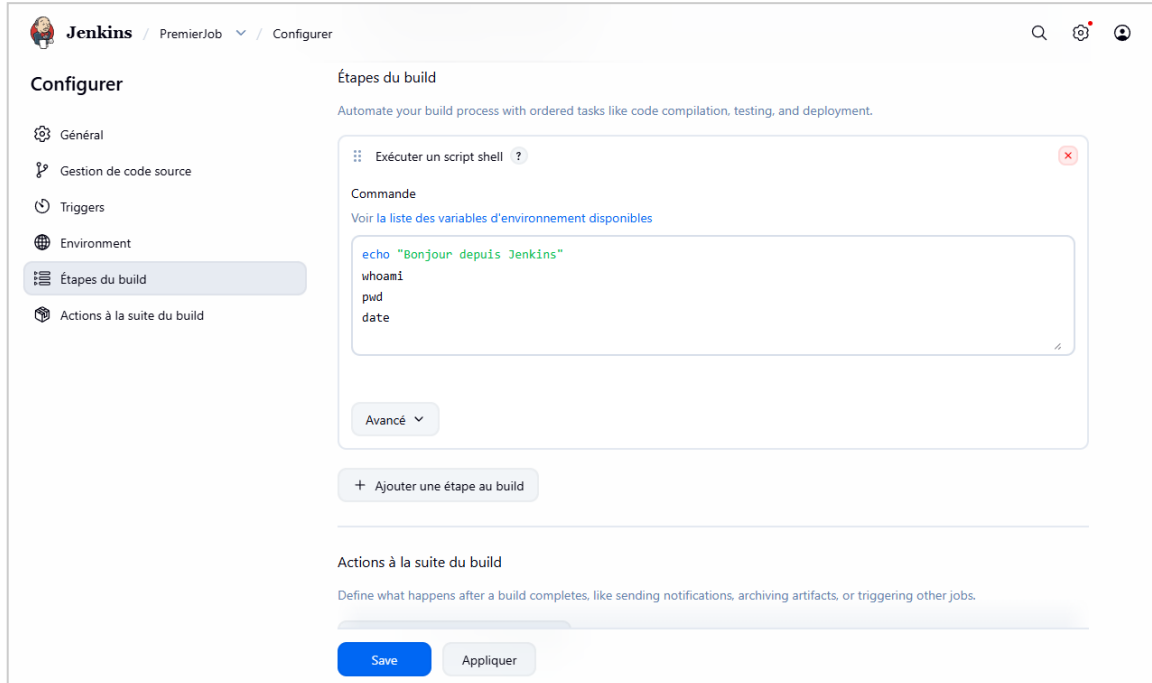
1. Sur le tableau de bord, cliquez sur **Nouveau Item**. Saisissez le nom `PremierJob`, choisissez **Construire un projet free-style**, puis cliquez sur **OK**.



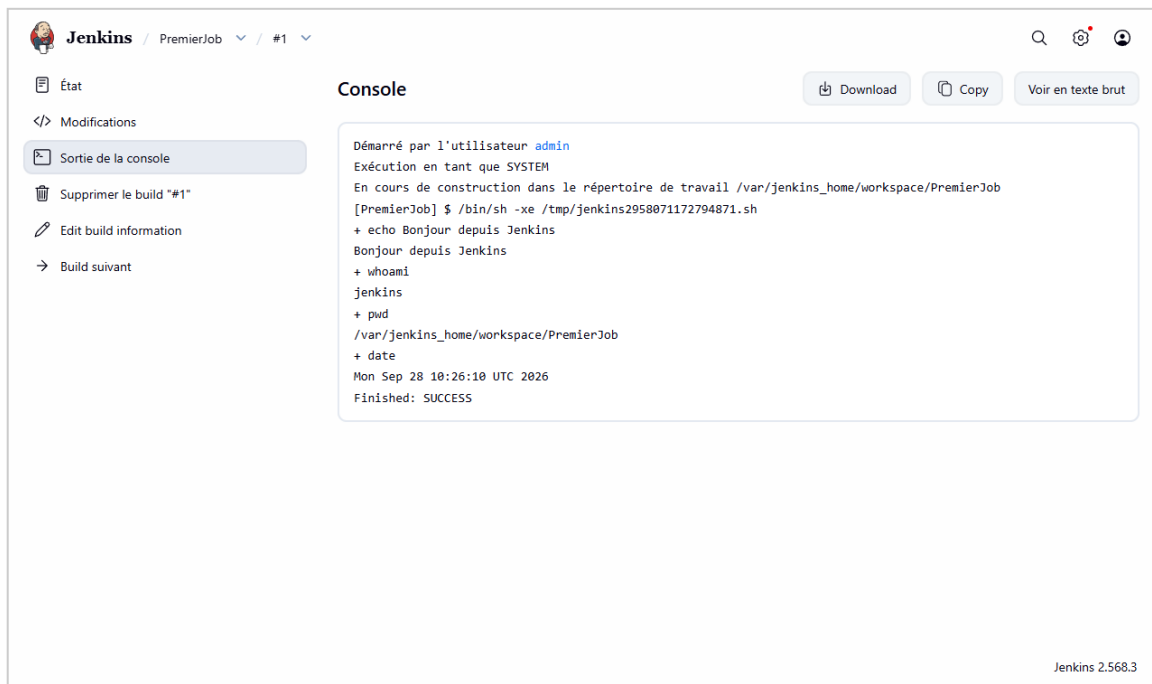
2. Dans la section **Étapes du build**, ajoutez une étape **Exécuter un script shell** avec le contenu

suivant, puis cliquez sur **Save** :

```
echo "Bonjour depuis Jenkins"
whoami
pwd
date
```



3. Cliquez sur **Lancer un build**. Un build **#1** apparaît dans l'historique : cliquez dessus, puis sur **Sortie de la console**.



4. **Provoquez un échec** : cliquez sur **Configure**, ajoutez une dernière ligne `exit 1` au script, sauvez et relancez un build. Le build **#2** est en échec : lisez sa sortie console.

The screenshot shows the Jenkins console for build #2 of 'PremierJob'. The console output is as follows:

```
Démarré par l'utilisateur admin
Exécution en tant que SYSTEM
En cours de construction dans le répertoire de travail /var/jenkins_home/workspace/PremierJob
[PremierJob] $ /bin/sh -xe /tmp/jenkins16594523001681337792.sh
+ echo Bonjour depuis Jenkins
Bonjour depuis Jenkins
+ whoami
jenkins
+ pwd
/var/jenkins_home/workspace/PremierJob
+ date
Mon Sep 28 10:26:11 UTC 2026
+ exit 1
Build step 'Exécuter un script shell' marked build as failure
Finished: FAILURE
```

Buttons at the top right: Download, Copy, Voir en texte brut.

Bottom right: Jenkins 2.568.3

5. **Réparez** : retirez la ligne `exit 1`, sauvez et relancez un build. Le build #3 est de nouveau un succès.

✓ Vérifier

L'historique du job montre un succès, un échec puis un succès.

The screenshot shows the Jenkins job overview for 'PremierJob'. The job status is 'Completed' (green checkmark). The 'Liens permanents' section lists the following builds:

- Dernier build (#3), il y a 10 s
- Dernier build stable (#3), il y a 10 s
- Dernier build avec succès (#3), il y a 10 s
- Dernier build en échec (#2), il y a 12 s
- Dernier build non réussi (#2), il y a 12 s
- Dernier build complété (#3), il y a 10 s

The 'Builds' section shows a list of builds:

Build	Status	Time
#3	Success	10:26
#2	Failure	10:26
#1	Success	10:26

Bottom right: REST API, Jenkins 2.568.3

```
./check.sh 2
```

✗ En cas de problème

Symptôme	Cause probable / solution
<code>check.sh</code> indique que le job n'existe pas	Le nom doit être exactement <code>PremierJob</code> (majuscules comprises). Vous pouvez renommer un job depuis sa page (Renommer).
Le build reste « en attente »	Aucun exécuteur disponible : attendez la fin des autres builds, ou vérifiez que Jenkins n'est pas en cours de redémarrage.

Q3) D'après la sortie console, sous quel utilisateur et dans quel dossier votre script a-t-il été exécuté ? Sur quelle machine ce dossier se trouve-t-il réellement ?

réponse 3 disponible.

Q4) Pourquoi la ligne `exit 1` suffit-elle à faire échouer le build ? Citez une commande « réelle » qui ferait échouer un build de la même manière.

réponse 4 disponible.

1.6. Récapitulatif

- ✔ Jenkins tourne dans un conteneur, sur votre machine, protégé par un compte administrateur.
- ✔ Vous savez créer un job, lancer un build et lire sa sortie console.
- ✔ Vous savez qu'un build échoue dès qu'une commande renvoie un code différent de `0`.

Au chapitre suivant, votre job va récupérer le code d'un vrai projet, le construire, le tester et le déployer.

2. Intégration continue de la calculatrice

Version 2.0.0 | Dernière mise à jour : 28/09/2026

Durée de réalisation : 6 heures

Auteur : Bauer Baptiste

Dans ce chapitre, vous allez mettre en place, **sur votre machine**, une chaîne d'intégration continue complète pour la calculatrice :

- Jenkins détecte chaque nouveau commit sur GitHub (étape 3) ;
- il construit l'image Docker, tests compris, et déploie l'application (étape 4) ;
- il vérifie que l'application répond, et refuse de déployer une version buguée (étape 5) ;
- toute la chaîne est décrite dans un fichier **Jenkinsfile** versionné avec le code (étape 6).

Prérequis

- Chapitre *Découvrir Jenkins* terminé : `./check.sh 1` et `./check.sh 2` validés.



Toutes les commandes de ce chapitre sont à lancer  **sur votre poste**, depuis la racine de votre dépôt **jenkins-calculatrice** (dans **Git Bash** sous Windows). Remplacez `<votre_compte_github>` par votre nom d'utilisateur GitHub (sensible à la casse).

2.1. Étape 3 : Relier un job au dépôt Git

🎯 **Objectif** : un job Jenkins qui récupère automatiquement la dernière version du code à chaque commit.

💡 Comprendre

Votre dépôt contient l'application :

Chemin	Contenu
<code>server.js</code>	Le serveur web (Express), qui écoute sur le port 3000 .
<code>src/calculatrice.js</code>	La fonction addition() .
<code>public/</code>	La page web (HTML, CSS, JavaScript).
<code>*.test.js</code>	Les tests automatisés (Jest), lancés par npm test .
<code>jenkins/</code> , <code>check.sh</code>	L'outillage du TP (chapitre précédent).

Pour savoir qu'il y a du nouveau code, Jenkins peut :

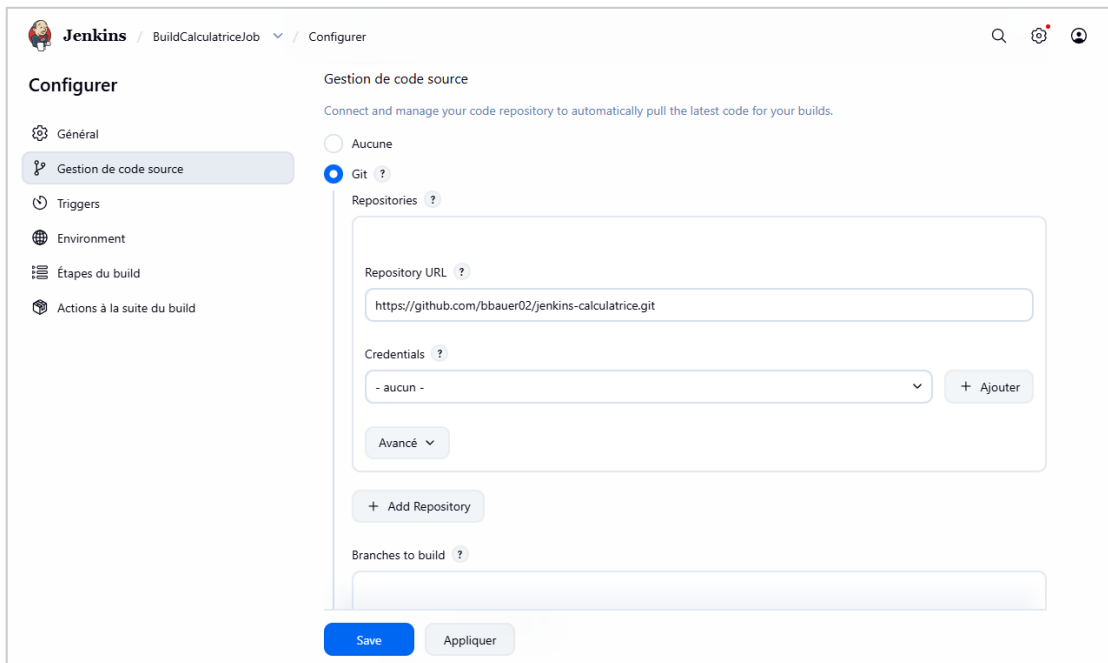
- **scruter** le dépôt (*polling*) : il demande régulièrement à GitHub « y a-t-il un nouveau commit ? » ;
- être **prévenu** par GitHub grâce à un **webhook**, à chaque **push**.

Le webhook est plus rapide et plus économe, mais GitHub doit pouvoir joindre Jenkins depuis Internet. Votre Jenkins tourne sur votre poste, qui n'est pas joignable depuis Internet : nous

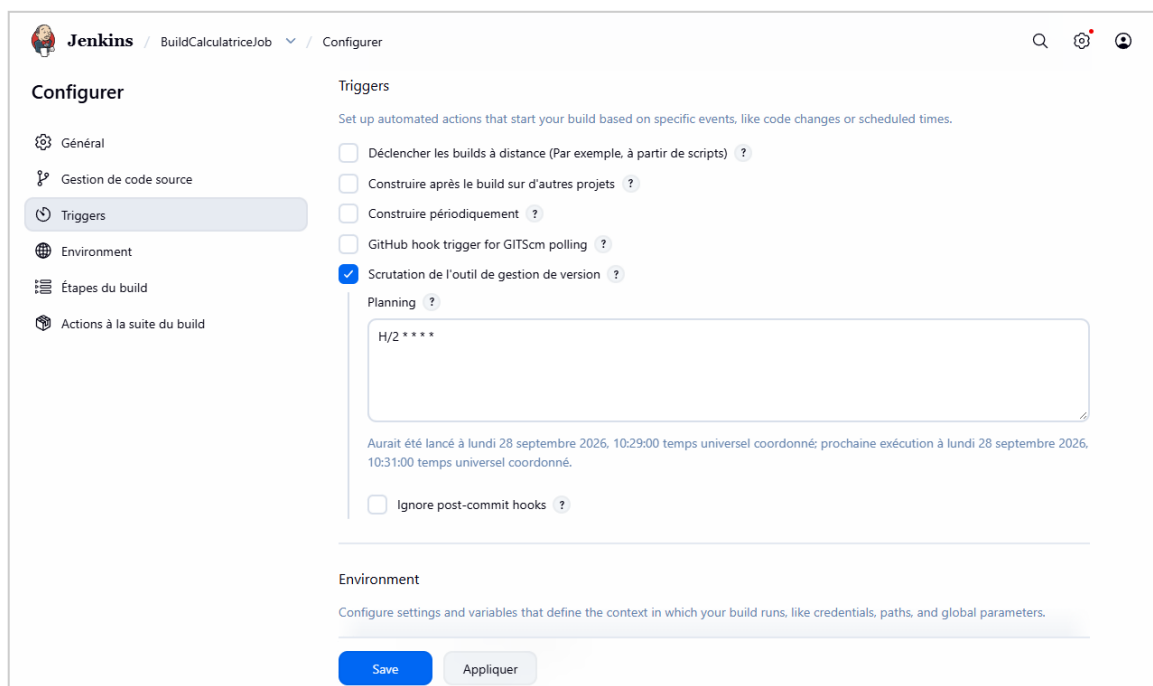
utilisons donc la **scrutation**. Le webhook viendra à l'étape 8, sur un serveur AWS.

🔧 À faire

1. Créez un nouveau job **free-style** nommé **BuildCalculatriceJob**.
2. Dans **Gestion de code source**, choisissez **Git** :
 - **Repository URL** : https://github.com/<votre_compte_github>/jenkins-calculatrice.git ;
 - **Credentials** : laissez - **aucun** - (votre dépôt est public) ;
 - **Branch Specifier** : **/main** (et non/master).



3. Dans **Triggers**, cochez **Scrutation de l'outil de gestion de version** et saisissez le planning **H/2 *** *
* * *





Le planning utilise la syntaxe **cron** : **minute heure jour mois jour-de-la-semaine**. **H/2 * * * *** signifie « toutes les 2 minutes environ ». Le **H** (*hash*) décale un peu l'horaire de chaque job pour éviter que tous interrogent GitHub au même instant.

4. Dans **Étapes du build**, ajoutez **Exécuter un script shell** :

```
ls
git log -1 --oneline
```

5. Sauvez, puis cliquez sur **Lancer un build**. Dans la sortie console, vous voyez Jenkins cloner votre dépôt, puis la liste des fichiers et le dernier commit.

6. **Testez la scrutation** : modifiez le fichier **README.md** (ajoutez une ligne), puis :

```
git commit -am "Test de la scrutation"
git push
```

Sans rien toucher, attendez au plus 2 minutes : un nouveau build démarre tout seul. Sa page indique « **Démarré par un changement dans la gestion de version** », et la sortie console affiche votre nouveau commit.

✓ **Vérifier**

```
./check.sh 3
```

✗ **En cas de problème**

Symptôme	Cause probable / solution
Couldn't find any revision to build	Le Branch Specifier ne correspond à aucune branche : utilisez */main .
repository not found / Authentication failed	URL mal saisie (casse du nom de compte), ou dépôt privé. Vérifiez l'URL en l'ouvrant dans votre navigateur.
Aucun build ne démarre après le push	Vérifiez que le push a bien eu lieu (git status doit indiquer up to date with 'origin/main'). Le menu Journal de scrutation du job montre ce que Jenkins a vu lors de sa dernière vérification.
check.sh indique que le dossier n'est pas un clone de votre dépôt	Vous travaillez dans le dépôt modèle au lieu du vôtre : clonez votre dépôt (étape 1).

Q5) Avec la scrutation **H/2 * * * ***, combien de requêtes Jenkins envoie-t-il à GitHub par jour, même s'il ne se passe rien ? Quel est l'avantage du webhook de ce point de vue ?

réponse 5 disponible.

2.2. Étape 4 : Construire et déployer avec Docker

🎯 **Objectif** : à chaque commit, Jenkins construit l'image Docker de l'application (tests compris) et relance le conteneur.

💡 Comprendre

Jenkins ne contient pas Node.js : il n'a pas besoin de savoir construire l'application lui-même. Il demande à **Docker** de le faire, à partir d'un **Dockerfile**. C'est le **Dockerfile** qui décrit comment installer les dépendances, lancer les tests et démarrer l'application.

Rappel : le démon Docker utilisé est celui de **votre machine**. Le conteneur de l'application sera donc créé à côté du conteneur Jenkins, et publié sur le port **3000** de votre machine.

🔧 À faire

Q6) Rédigez à la racine du dépôt un **Dockerfile multi-stage** pour l'application :

- un premier stage nommé **builder** part de **node:22-alpine**, installe toutes les dépendances (**npm ci**), copie le code et exécute les tests (**npm test**) ;
- un second stage, également basé sur **node:22-alpine**, n'installe que les dépendances de production (**npm ci --omit=dev**), copie depuis **builder** les fichiers **server.js**, **src/** et **public/**, s'exécute avec l'utilisateur non-root **node** et lance **node server.js** sur le port **3000**.

réponse 6 disponible.

Q7) Rédigez à la racine du dépôt un fichier **docker-compose.yaml** qui :

- fixe le nom du projet Compose à **calculatrice** (clé **name**) ;
- construit l'image **calculatrice:latest** à partir du **Dockerfile** ;
- nomme le conteneur **calculatrice-app** et publie le port **3000** ;
- redémarre automatiquement le conteneur (**restart: unless-stopped**).

réponse 7 disponible.



Le fichier **.dockerignore**, déjà présent dans le dépôt, évite de copier dans l'image les fichiers inutiles (**node_modules**, **.git**, **jenkins/...**). Il ne doit **pas** exclure les fichiers ***.test.js** : ils sont exécutés par **npm test** pendant la construction.

1. 🖥 Avant de confier ce travail à Jenkins, vérifiez vous-même que tout fonctionne :

```
docker compose up -d --build
```

Pendant la construction, vous devez voir les tests passer (Tests: 20 passed, 20 total). Ouvrez ensuite <http://localhost:3000> : la calculatrice s'affiche. Arrêtez-la avant de continuer :

```
docker compose down
```

2.  Créez le script `calculatrice-app.sh` que Jenkins exécutera :

calculatrice-app.sh

```
#!/bin/bash
# Construit l'image (tests compris) et (re)lance l'application sur le port 3000
set -e

echo "*****"
echo "CONSTRUCTION DE L'APPLICATION SUR LE PORT 3000"
docker compose up -d --build
echo "*****"
date
```



- `set -e` arrête le script (et fait échouer le build) dès qu'une commande échoue.
- On écrit `docker compose` (le plugin Compose, installé dans l'image Jenkins) et non `docker-compose` (l'ancien programme Compose v1, abandonné) : sinon, le build échoue avec `docker-compose: not found`.

3.  Versionnez et poussez les trois fichiers :

```
git add Dockerfile docker-compose.yml calculatrice-app.sh
git commit -m "Ajout du Dockerfile, du docker-compose et du script de build"
git push
```

```
PS C:\Users\baptiste\Documents\Cours\tests_unitaires\calculatrice-jenkins> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .dockerignore
        Dockerfile
        docker-compose.yml

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\baptiste\Documents\Cours\tests_unitaires\calculatrice-jenkins> |
```

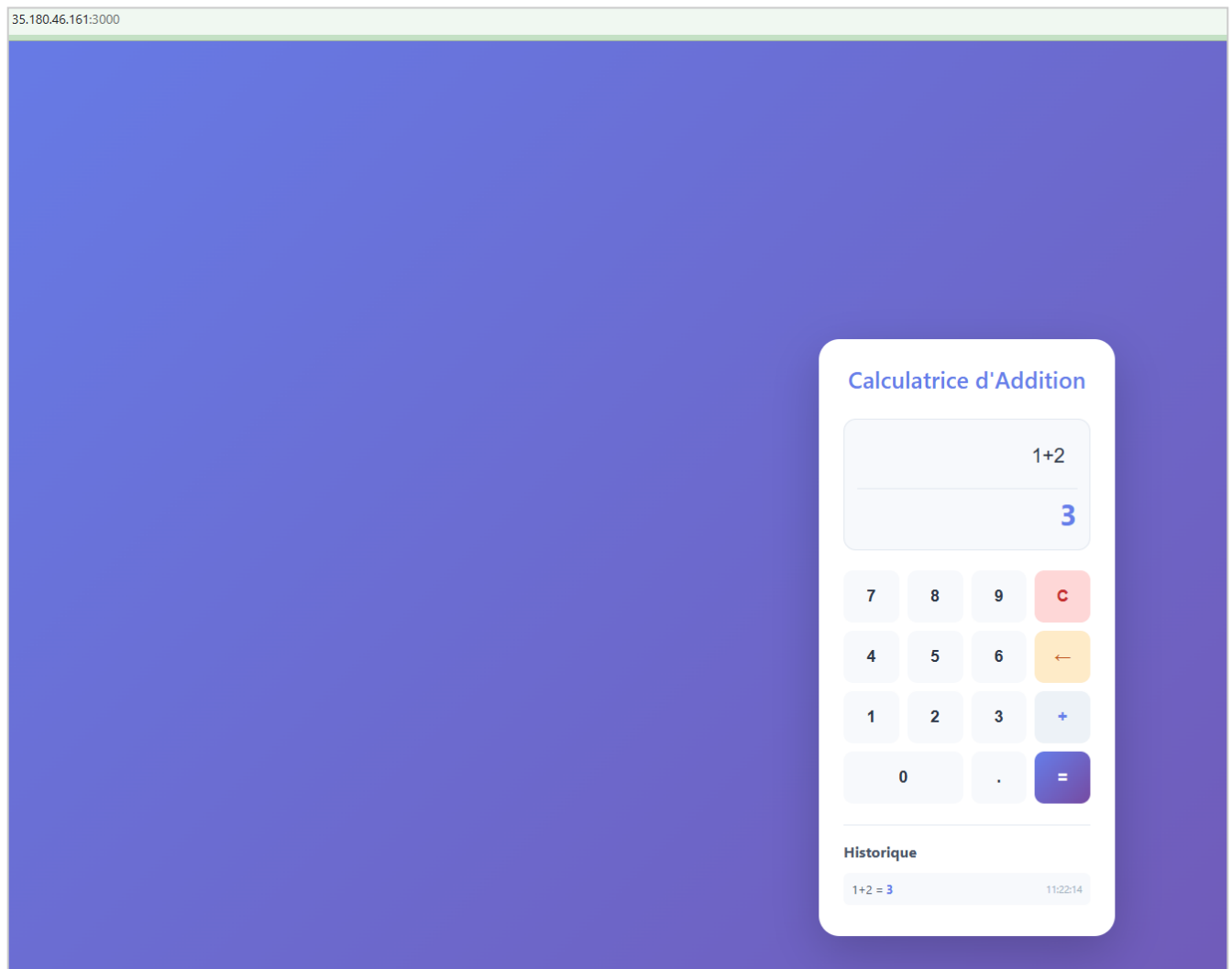
4. Dans Jenkins, configurez `BuildCalculatriceJob` : remplacez le script de l'étape du build par :

```
bash ./calculatrice-app.sh
```

5. Lancez un build (ou attendez la scrutation). La sortie console montre la construction de l'image, les tests, puis le démarrage du conteneur.

✓ Vérifier

L'application est accessible sur <http://localhost:3000>.



```
./check.sh 4
```

✗ En cas de problème

Symptôme	Cause probable / solution
no configuration file provided: not found	Pas de <code>docker-compose.yaml</code> à la racine du dépôt sur GitHub : vérifiez le nom du fichier et qu'il a bien été poussé.
docker-compose: not found	Remplacez <code>docker-compose</code> par <code>docker compose</code> dans le script.
No tests found pendant npm test	Les fichiers <code>*.test.js</code> sont exclus par le <code>.dockerignore</code> , ou ne sont pas copiés dans le stage <code>builder</code> .

Symptôme	Cause probable / solution
<code>container name "/calculatrice-app" is already in use</code>	Un conteneur du même nom a été créé par un autre projet Compose. Ajoutez <code>name: calculatrice</code> dans <code>docker-compose.yaml</code> , ou supprimez-le : <code>docker rm -f calculatrice-app</code> .
<code>port is already allocated</code> (port 3000)	Une autre application utilise le port 3000 (par exemple un <code>npm start</code> oublié). Arrêtez-la.



Bloqué ? Récupérez la correction de l'étape avec la branche `solution-etape-4` (voir *Comment utiliser ce cours*).

Q8) Pourquoi est-il intéressant de lancer les tests **pendant la construction de l'image** plutôt qu'après son déploiement ?

réponse 8 disponible.

2.3. Étape 5 : Tester, casser, réparer

🎯 **Objectif** : vérifier automatiquement que l'application répond après chaque déploiement, et voir la chaîne bloquer une version buguée.

2.3.1. Un job qui teste l'application

💡 Comprendre

Les tests unitaires vérifient le code **avant** le déploiement. Un **smoke test** (« test de fumée ») vérifie **après** le déploiement que l'application démarre et répond. Nous allons l'exécuter dans un second job, déclenché automatiquement après chaque build réussi.

🔧 À faire

1. Créez un job **free-style** nommé `TestCalculatriceJob`, **sans** gestion de code source.
2. Dans **Triggers**, cochez **Construire après le build sur d'autres projets**, projet à surveiller : `BuildCalculatriceJob`, option **Déclencher que si la construction est stable**.

The screenshot shows the Jenkins configuration interface for a job named 'TestCalculatriceJob'. The left sidebar contains a menu with options: 'Général', 'Gestion de code source', 'Triggers' (selected), 'Environment', 'Étapes du build', and 'Actions à la suite du build'. The main area is titled 'Configurer' and has a sub-section 'Triggers'. It includes a description: 'Set up automated actions that start your build based on specific events, like code changes or scheduled times.' There are several checkboxes: 'Déclencher les builds à distance (Par exemple, à partir de scripts)' is unchecked; 'Construire après le build sur d'autres projets' is checked. Below this, there is a field 'Projet à surveiller' containing 'BuildCalculatriceJob'. There are four radio buttons for triggering conditions: 'Déclencher que si la construction est stable' (selected), 'Déclencher même si la construction est instable', 'Déclencher même si la construction échoue', and 'Always trigger, even if the build is aborted'. There are also three unchecked checkboxes: 'Construire périodiquement', 'GitHub hook trigger for GITScm polling', and 'Scrutation de l'outil de gestion de version'. At the bottom, there is an 'Environment' section with a description: 'Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.' It has one unchecked checkbox: 'Delete workspace before build starts'. At the very bottom are two buttons: 'Save' and 'Appliquer'.

3. Dans **Étapes du build**, ajoutez **Exécuter un script shell** :

```
curl -fsS --retry 10 --retry-connrefused --retry-delay 3
http://host.docker.internal:3000 | grep "Calculatrice d'Addition"
```

- `host.docker.internal` désigne votre machine, vue depuis le conteneur Jenkins : c'est là que l'application publie son port `3000`. (`localhost` désignerait le conteneur Jenkins lui-même.)
- `--retry 10 --retry-connrefused --retry-delay 3` : l'application peut mettre quelques secondes à démarrer ; `curl` réessaie jusqu'à 10 fois, toutes les 3 secondes.
- `-f` : `curl` échoue si le serveur renvoie une erreur HTTP (404, 500...).
- `grep` cherche le titre de la page. S'il ne le trouve pas, il renvoie un code d'erreur et le build échoue.

4. Lancez `BuildCalculatriceJob` : à la fin du build, `TestCalculatriceJob` démarre tout seul et doit réussir.

2.3.2. Casser le build

🔧 À faire

1. 🧠 **Avant de faire la manipulation, notez votre prédiction** : si vous poussez un bug dans la fonction `addition()`, que va-t-il se passer pour le build ? pour le test ? pour l'application actuellement en ligne ?
2. 💻 Introduisez un bug : dans `src/calculatrice.js`, remplacez `return nombre1 + nombre2;` par `return nombre1 - nombre2;`, puis :

```
git commit -am "Nouvelle version de l'addition"
git push
```

3. Attendez le build (ou lancez-le). Il est **en échec** : dans la sortie console, cherchez la ligne **Tests:** et le détail des tests en échec (**Expected: 5, Received: -1**).



```
#11 [builder 6/6] RUN npm test
#11 0.632
#11 0.632 > calculatrice-addition@1.0.0 test
#11 0.632 > jest
#11 0.632
#11 3.157 FAIL src/calculatrice.test.js
#11 3.157   ● Module Calculatrice > addition() > devrait additionner deux nombres entiers positifs
#11 3.157
#11 3.157     expect(received).toBe(expected) // Object.is equality
#11 3.157
#11 3.157     Expected: 5
#11 3.157     Received: -1
#11 3.157
#11 3.157     5 | | // Tests avec des nombres entiers
#11 3.157     6 | test('devrait additionner deux nombres entiers positifs', () => {
#11 3.157   > 7 |   expect(addition(2, 3)).toBe(5);
#11 3.157     |   ^
#11 3.157     8 |   expect(addition(10, 20)).toBe(30);
#11 3.157     9 |   expect(addition(0, 0)).toBe(0);
#11 3.157    10 | });
#11 3.157
#11 3.157     at Object.toBe (src/calculatrice.test.js:7:36)
#11 3.157
#11 3.157   ● Module Calculatrice > addition() > devrait additionner des nombres négatifs
#11 3.157
#11 3.157     expect(received).toBe(expected) // Object.is equality
#11 3.157
#11 3.157     Expected: -8
#11 3.157     Received: -2
#11 3.157
```

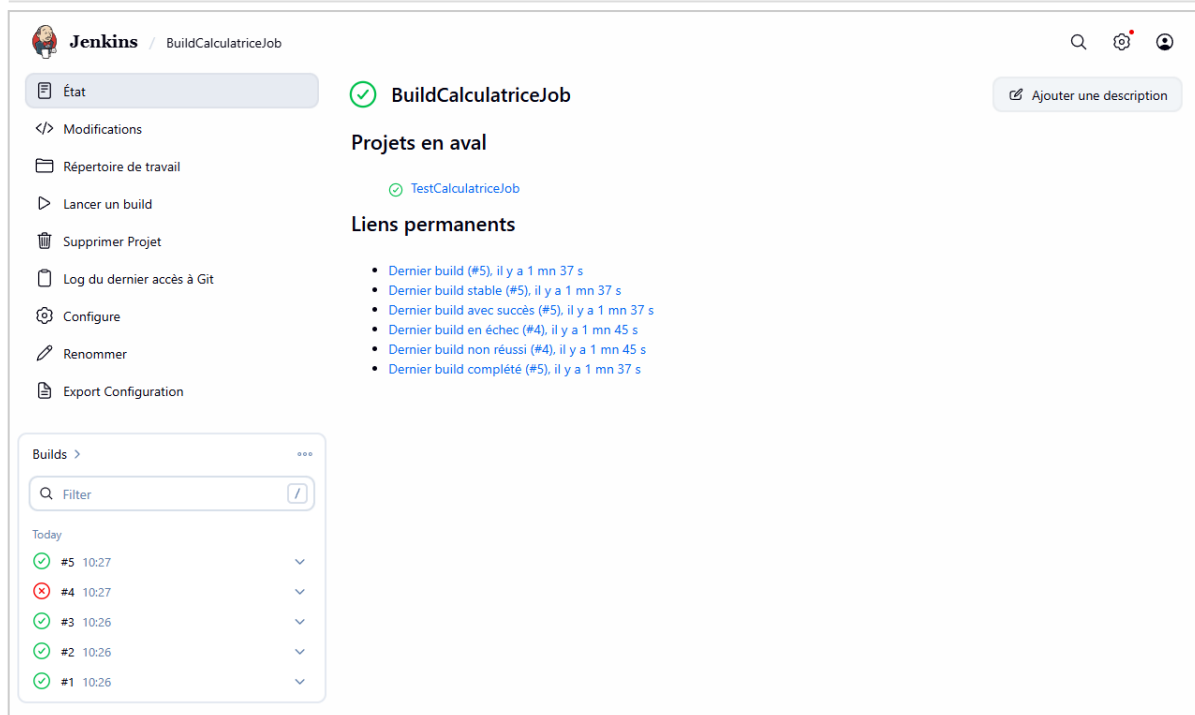
4. Observez :
 - **TestCalculatriceJob** n'a **pas** été déclenché (le build n'est pas stable) ;
 - <http://localhost:3000> fonctionne toujours et calcule **juste** : c'est l'ancienne version, la version buguée n'a jamais été déployée.

2.3.3. Réparer

1. 🖥️ Annulez le commit fautif :

```
git revert --no-edit HEAD
git push
```

2. Le build suivant redevient vert, et **TestCalculatriceJob** est de nouveau déclenché.



✓ Vérifier

```
./check.sh 5
```

✗ En cas de problème

Symptôme	Cause probable / solution
<code>curl: (7) Failed to connect to host.docker.internal</code>	L'application n'a pas démarré (<code>docker logs calculatrice-app</code>), ou le conteneur Jenkins n'a pas été lancé avec le <code>compose.yaml</code> fourni (option <code>extra_hosts</code>).
<code>curl: (6) Could not resolve host: host.docker.internal</code>	Même cause : relancez Jenkins avec <code>docker compose up -d</code> dans le dossier <code>jenkins/</code> .
Le build ne casse pas	Vérifiez que le bug a été poussé (<code>git log origin/main -1</code>), et que le <code>Dockerfile</code> exécute bien <code>npm test</code> .
<code>TestCalculatriceJob</code> ne se lance jamais	Vérifiez le nom du projet surveillé dans ses Triggers : <code>BuildCalculatriceJob</code> , exactement.

Q9) Comparez votre prédiction avec ce que vous avez observé. Pourquoi l'application en ligne a-t-elle continué à fonctionner correctement pendant que le build était rouge ?

réponse 9 disponible.

2.4. Étape 6 : Pipeline et Jenkinsfile

2.4.1. 6.1 Enchaîner les jobs dans un pipeline

🎯 **Objectif** : découvrir les pipelines Jenkins, qui décrivent une chaîne d'étapes (*stages*) par un script.

💡 Comprendre

Nos deux jobs free-style s'enchaînent grâce à un déclencheur. Un **pipeline** décrit toute la chaîne dans un script, découpé en **stages**, dont Jenkins affiche l'avancement.

🔧 À faire

1. Créez un job de type **Pipeline** nommé **CalculatricePipeline**.
2. Dans la section **Pipeline** (*Definition : Pipeline script*), saisissez :

```
node {
  stage('Preparation') {
    // Supprime l'ancien conteneur s'il existe ("|| true" : pas d'erreur s'il
    n'existe pas)
    sh 'docker rm -f calculatrice-app || true'
  }
  stage('Build') {
    build 'BuildCalculatriceJob'
  }
  stage('Results') {
    build 'TestCalculatriceJob'
  }
}
```

3. Sauvez et lancez un build : la page du job affiche les trois stages et leur durée.



Vous remarquerez que **TestCalculatriceJob** s'exécute **deux fois** : une fois parce que le pipeline le lance (stage **Results**), une fois grâce à son propre déclencheur. C'est une limite des pipelines qui enchaînent des jobs free-style. La solution : décrire toute la chaîne **dans le pipeline lui-même**.

2.4.2. 6.2 Un Jenkinsfile de CI complet

🎯 **Objectif** : décrire toute la chaîne dans un fichier **Jenkinsfile**, versionné avec le code.

💡 Comprendre

Un **Jenkinsfile** placé à la racine du dépôt est lu par Jenkins à chaque exécution. Le pipeline est ainsi **versionné** : il évolue avec le code, se relit dans une *pull request* et se restaure avec Git.

Table 2. Anatomie d'un pipeline déclaratif

Bloc	Rôle
<code>pipeline { }</code>	Englobe toute la description.

Bloc	Rôle
<code>agent any</code>	Exécute le pipeline sur n'importe quel exécuteur disponible (ici, Jenkins lui-même).
<code>triggers { }</code>	Les déclencheurs, par exemple <code>pollSCM('H/2 * * * *')</code> .
<code>options { }</code>	Réglages, par exemple <code>disableConcurrentBuilds()</code> pour éviter deux déploiements simultanés.
<code>environment { }</code>	Variables disponibles dans toutes les étapes (<code>\$APP_URL</code>).
<code>stages { stage('Nom') { steps { ... } } }</code>	Les étapes de la chaîne, exécutées dans l'ordre. Le pipeline s'arrête au premier stage en échec.
<code>post { success { } failure { } always { } }</code>	Actions exécutées à la fin, selon le résultat.

Jenkins fournit aussi des variables d'environnement, par exemple `GIT_COMMIT` (le hash du commit construit) et `BUILD_NUMBER`.

🔧 À faire

Q10) Créez à la racine du dépôt un fichier `Jenkinsfile` à partir du squelette ci-dessous, en remplaçant chaque `TODO` par la ou les commandes `sh '...'` adaptées.

```

pipeline {
    agent any
    triggers {
        pollSCM('H/2 * * * *')
    }
    options {
        disableConcurrentBuilds()
    }
    environment {
        APP_URL = 'http://host.docker.internal:3000'
    }
    stages {
        stage('Tests') {
            steps {
                // TODO : construire uniquement le stage "builder" du Dockerfile
                (il exécute npm test)
            }
        }
        stage('Build') {
            steps {
                // TODO : construire l'image de l'application avec Docker Compose
                // TODO : ajouter à l'image calculatrice:latest un tag égal aux 7
                premiers caractères de GIT_COMMIT
            }
        }
        stage('Déploiement') {
    
```

```
        steps {
            // TODO : (re)lancer l'application avec Docker Compose
        }
    }
    stage('Smoke test') {
        steps {
            // TODO : vérifier que $APP_URL répond et contient "Calculatrice"
        }
    }
}
post {
    success {
        echo "Version ${env.GIT_COMMIT.substring(0, 7)} déployée sur le port
3000"
    }
    failure {
        sh 'docker compose logs --tail=50 || true'
    }
    always {
        sh 'docker image prune -f || true'
    }
}
}
```

▼ Indices

- Construire un seul stage d'un Dockerfile : `docker build --target <stage> -t <nom:tag> .`
- Construire les images d'un projet Compose : `docker compose build` ; les lancer : `docker compose up -d`.
- Dans une chaîne Groovy entre guillemets doubles, `${env.GIT_COMMIT.substring(0, 7)}` est remplacé par le début du hash.
- Le smoke test est la commande de `TestCalculatriceJob`, avec `$APP_URL` à la place de l'adresse (entre guillemets simples, c'est le shell qui remplace `$APP_URL`).

réponse **10** disponible.

1.  Pousser le `Jenkinsfile` :

```
git add Jenkinsfile
git commit -m "Pipeline de CI complet dans un Jenkinsfile"
git push
```

2. Dans Jenkins, configurez `CalculatricePipeline` → section **Pipeline** :

- **Definition** : `Pipeline script from SCM` ;
- **SCM** : `Git`, **Repository URL** : l'URL de votre dépôt, **Credentials** : aucun ;
- **Branch Specifier** : `*/main` ;

- **Script Path : Jenkinsfile.**

- Désactivez les anciens jobs, pour qu'un seul job déploie l'application : ouvrez **BuildCalculatriceJob** → **Configure**, basculez l'interrupteur **Enabled** (en haut à droite) sur désactivé, puis **Save**. Faites de même pour **TestCalculatriceJob**.
- Lancez **une fois à la main CalculatricePipeline**. Ce premier build enregistre le déclencheur **pollSCM** écrit dans le **Jenkinsfile** : les suivants démarreront tout seuls après chaque push.

✓ Vérifier

Les quatre stages sont verts, et la sortie console affiche, dans la partie **post**, la ligne **Version xxxxxxxx déployée sur le port 3000** (les 7 premiers caractères du commit).

Stage	Declarative: Checkout SCM	Tests	Build	Déploiement	Smoke test	Declarative: Post Actions
Average stage times: (full run time: ~24s)	1s	2s	2s	12s	439ms	767ms
#1 sept. 28 12:27 No Changes	1s	2s	2s	12s	439ms	767ms

Liens permanents

- Dernier build (#1), il y a 29 s
- Dernier build stable (#1), il y a 29 s
- Dernier build avec succès (#1), il y a 29 s
- Dernier build complété (#1), il y a 29 s

REST API Jenkins 2.568.3

```
./check.sh 6
```

✖ En cas de problème

Symptôme	Cause probable / solution
WorkflowScript: ... expecting '}' (erreur au démarrage)	Erreur de syntaxe dans le Jenkinsfile : accolade ou guillemet manquant. Le numéro de ligne est indiqué dans le message.
No steps specified for branch / stage vide	Un TODO n'a pas été remplacé : chaque bloc steps doit contenir au moins une commande.
target stage "builder" could not be found	Le premier stage de votre Dockerfile ne s'appelle pas builder (FROM node:22-alpine AS builder).
Scripts not permitted to use method ...	Utilisez <code>substring(0, 7)</code> pour tronquer le hash, comme dans les indices.



Bloqué ? Récupérez la correction de l'étape avec la branche **solution-etape-6**.

2.4.3. 6.3 Exercice de débogage (approfondissement)

🎯 **Objectif** : savoir diagnostiquer un pipeline en échec à partir de sa sortie console.

Le dépôt modèle contient un **Jenkinsfile** qui comporte **trois erreurs**.

1. 🖥 Remplacez votre **Jenkinsfile** par la version à déboguer :

```
git fetch https://github.com/bbauer02/jenkins-calculatrice.git exercice-debug
git checkout FETCH_HEAD -- Jenkinsfile
git commit -m "Exercice : Jenkinsfile à déboguer"
git push
```

2. Lancez le pipeline. Repérez le stage en rouge, lisez la sortie console, corrigez, poussez, **relancez à la main...** jusqu'à ce que le pipeline soit vert.



Ce **Jenkinsfile** ne contient **pas** de bloc **triggers** : dès sa première exécution, Jenkins retire la scrutation de la configuration du job. Pendant l'exercice, les push ne déclenchent donc plus rien : relancez le pipeline à la main après chaque correction.

3. Une fois le pipeline vert, remettez le **Jenkinsfile** complet de l'étape 6.2 (celui de la correction) :

```
git fetch https://github.com/bbauer02/jenkins-calculatrice.git solution-etape-6
git checkout FETCH_HEAD -- Jenkinsfile
git commit -m "Retour au Jenkinsfile complet"
git push
```

Lancez ensuite **une fois à la main** `CalculatricePipeline` : ce build réenregistre le déclencheur `pollSCM`, et les push suivants redémarrent le pipeline tout seuls.



Le pipeline s'arrête au premier stage en échec : les erreurs apparaissent une par une. Après chaque correction, relancez le pipeline pour découvrir la suivante.

Q11) Pour chacune des trois erreurs, indiquez le stage en échec, le message qui vous a permis de la trouver et la correction.

réponse 11 disponible.

2.4.4. 6.4 Mesurer la couverture de tests (défi)

Q12) Ajoutez au `Jenkinsfile`, juste après le stage `Tests`, un stage `Couverture` qui affiche le taux de couverture du code par les tests. Le script `npm run test:coverage` existe dans `package.json`, et l'image `calculatrice:test` contient tout le nécessaire.

réponse 12 disponible.

2.5. Récapitulatif

- ✔ Jenkins détecte chaque commit sur votre dépôt (scrutation).
- ✔ L'image est construite et testée par Docker ; une version buguée n'est jamais déployée.
- ✔ Un smoke test vérifie que l'application répond après le déploiement.
- ✔ Toute la chaîne est décrite dans un `Jenkinsfile` versionné avec le code.

Il reste une limite : tout tourne sur votre poste. Dans les chapitres suivants, vous allez installer la même chaîne sur un serveur AWS, et GitHub préviendra Jenkins à chaque push grâce à un webhook.

3. Préparer une instance AWS EC2

Version 2.0.0 | Dernière mise à jour : 28/09/2026

Durée de réalisation : 2 heures

Auteur : Bauer Baptiste

3.1. Étape 7 : Préparer un serveur AWS EC2

🎯 **Objectif** : disposer d'un serveur Linux sur Internet, accessible en SSH, avec Docker installé.

💡 Comprendre

Jusqu'ici, Jenkins tournait sur votre poste. C'est pratique pour apprendre, mais :

- votre poste n'est pas joignable depuis Internet : GitHub ne peut pas prévenir Jenkins (webhook) ;
- l'application n'est disponible que lorsque votre ordinateur est allumé.

Dans ce chapitre, vous allez donc préparer un **serveur** :

- créer une paire de clés SSH et une instance **AWS EC2** ;
- configurer votre poste pour vous y connecter en **SSH** ;
- installer **Docker Engine** sur l'instance.

Au chapitre suivant, vous y installerez la même chaîne que sur votre poste.

Prérequis

- Chapitre *Intégration continue de la calculatrice* terminé (./check.sh 6 validé).
- Un compte AWS (ou AWS Academy) avec accès à la console EC2.

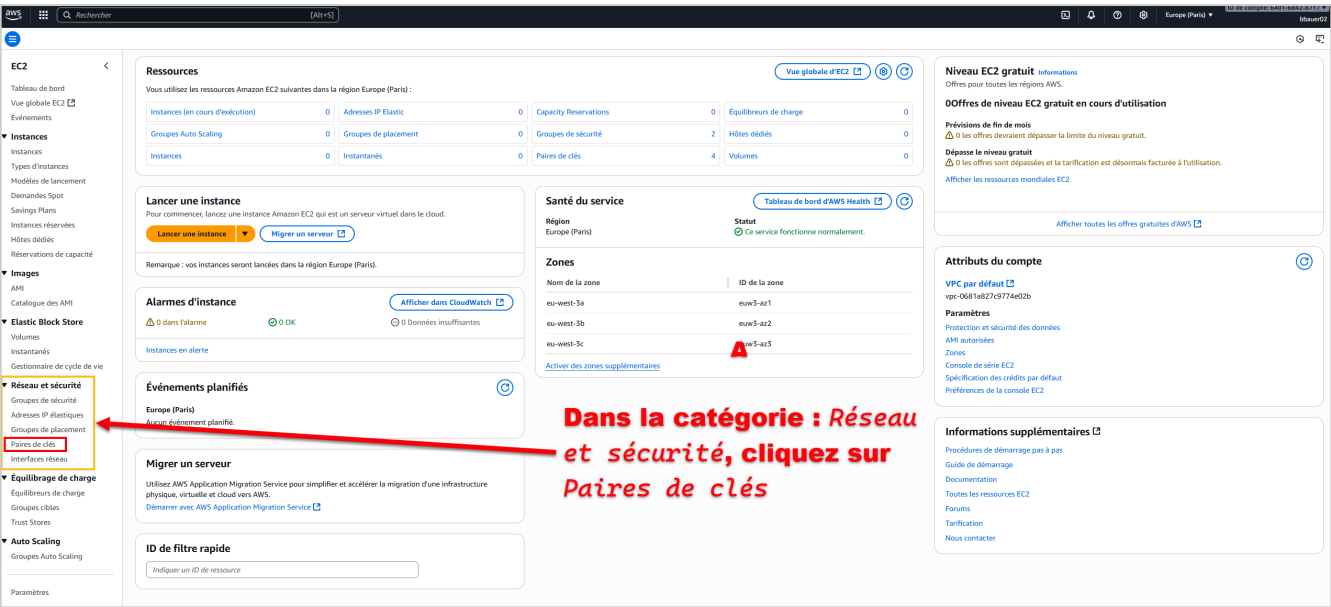


Où taper les commandes ? 🖥️ **Poste local** : votre ordinateur. ☁️ **Instance EC2** : le serveur distant, une fois connecté en SSH.

3.1.1. Création de la paire de clés SSH

Pour se connecter à une instance EC2 via SSH, vous devez d'abord créer une paire de clés SSH. Voici comment le faire :

Dans le panneau de contrôle AWS, allez dans la section "EC2" puis "Key Pairs" sous "Network & Security".



Cliquez sur **Create Key Pair**, donnez un nom à votre paire de clés, choisissez le type de clé (voir le comparatif ci-dessous) et le format **PEM** pour une utilisation avec OpenSSH. Cliquez ensuite sur "Create Key Pair" pour télécharger le fichier **.pem**.

- **RSA** : ancien standard, très répandu, fiable, mais lourd (clés et signatures volumineuses, calculs plus lents).
- **ED25519** : moderne, rapide, signatures compactes, sécurité forte, mais parfois moins compatible avec les environnements anciens.

🔑 Pour SSH : **ED25519 est recommandé par défaut**, sauf si vous devez gérer des systèmes anciens qui ne le supportent pas.

Table 3. Comparatif RSA vs ED25519

Critère	RSA	ED25519
Type d’algorithme	Basé sur la factorisation de grands nombres premiers	Basé sur les courbes elliptiques (Edwards Curve 25519)
Sécurité équivalente	RSA 2048 ≈ 112 bits RSA 4096 ≈ 150 bits	Clé 256 bits ≈ 128 bits (équivalent RSA 3072)
Taille de clé	Grande (2048–4096 bits)	Petite (256 bits)
Performance	Plus lent (génération, signature, vérification) Lourd sur petits appareils	Très rapide, constant-time (protège contre attaques par timing)
Conception	Ancien, éprouvé, universellement supporté	Moderne, conçu pour éviter les erreurs d’implémentation
Compatibilité	Supporté partout (anciens systèmes, logiciels propriétaires)	Supporté par OpenSSH >= 6.5 (2014), moins universel sur vieux systèmes
Taille des signatures	Grandes (plusieurs centaines d’octets)	Petites (64 octets)

Critère	RSA	ED25519
Simplicité d'usage	Paramètres nombreux (taille de clé, padding) → risques de mauvaise config	Safe by default, peu de paramètres

Créer une paire de clés [Informations](#)

Paire de clés
Une paire de clés, composée d'une clé privée et d'une clé publique, est un ensemble d'informations d'identification de sécurité que vous utilisez pour prouver votre identité lors de la connexion à une instance.

Nom

Le nom peut avoir un maximum de 255 caractères ASCII. Il ne peut pas inclure d'espaces avant ou après.

Type de paire de clés [Informations](#)
☐ RSA
 ☒ ED25519

Format de fichier de clé privée
☒ .pem
À utiliser avec OpenSSH
☐ .ppk
À utiliser avec PuTTY

Balises - facultatif
Aucune balise n'est associée à cette ressource.

[Ajouter une balise](#)
Vous pouvez ajouter jusqu'à 50 identifications supplémentaires.

Choisissez .pem pour une utilisation avec votre client SSH OpenSSH

[Annulez](#) [Créer une paire de clés](#)

Quand vous cliquez sur "Create Key Pair", le fichier **.pem** est automatiquement téléchargé.



Ce fichier contient votre **clé privée**. AWS ne la conserve pas : si vous la perdez, vous ne pourrez plus vous connecter à l'instance.

- Ne la partagez jamais (ni par e-mail, ni sur Discord, ni dans un dépôt Git).
- Conservez-la dans un répertoire protégé (voir plus bas).

3.1.2. Création de l'instance EC2

Dans la console AWS, section EC2 → **Instances** → **Launch instances**, renseignez :

Paramètre	Valeur conseillée
Name	jenkins-server (ou un nom de votre choix)
AMI	Ubuntu Server 24.04 LTS (les commandes du cours sont écrites pour Ubuntu)
Instance type	t3.medium (2 vCPU, 4 Go de RAM) au minimum. Jenkins et un build Node.js saturent une t2.micro / t3.micro (1 Go).
Key pair	La paire de clés créée à l'étape précédente
Network settings	Créer un Security Group avec une règle entrante SSH (22) limitée à My IP
Storage	20 à 30 Go (gp3) : les images Docker prennent vite de la place

Cliquez sur **Launch instance**.



Adresse IP publique et coûts

- L'adresse IP publique d'une instance **change chaque fois qu'on l'arrête puis la redémarre** (*Stop* puis *Start* ; un simple *Reboot* la conserve). Or nous allons la déclarer dans GitHub (webhook). Pour qu'elle reste stable, associez une **Elastic IP** à l'instance (EC2 → Elastic IPs → **Allocate** puis **Associate**).
- Une `t3.medium` n'est **pas** couverte par l'offre gratuite. Pensez à **arrêter l'instance** à la fin de chaque séance (voir la dernière section de ce chapitre).
- Depuis février 2024, AWS facture toute adresse IPv4 publique, Elastic IP comprise (quelques dixièmes de centime par heure), y compris lorsque l'instance est arrêtée.

3.1.3. Installer et vérifier le client SSH

Sur la plupart des systèmes Unix (Linux, macOS), SSH est préinstallé. Pour Windows, vous pouvez utiliser PowerShell ou le **Terminal Windows**, que je conseille d'installer.

3.1.3.1. Vérifier si SSH est installé

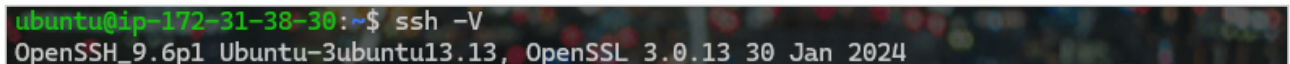
-  **Poste local** : ouvrez un terminal et tapez la commande suivante :

```
ssh -V
```



```
PS C:\Users\baptiste> ssh -V
OpenSSH for Windows 9.5p1, LibreSSL 3.8.2
```

Figure 2. Sur Windows



```
ubuntu@ip-172-31-38-30:~$ ssh -V
OpenSSH_9.6p1 Ubuntu-3ubuntu13.13, OpenSSL 3.0.13 30 Jan 2024
```

Figure 3. Sur Ubuntu

Si la commande affiche une version d'OpenSSH, passez directement à la section suivante. Sinon, installez le client :

3.1.3.2. Installer un client OpenSSH sur Windows

-  Dans un **PowerShell ouvert en mode administrateur**, exécutez la commande suivante pour installer le client OpenSSH :

```
Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0
```

3.1.3.3. Installer un client OpenSSH sur Linux

-  Dans un terminal, exécutez la commande suivante pour installer le client OpenSSH :

```
sudo apt update && sudo apt install -y openssh-client
```

3.1.4. Configurer les permissions de la clé privée

- Déplacez le fichier `.pem` dans un répertoire sécurisé de votre poste local : `~/.ssh/` sur Linux et macOS, ou `C:\Users\VotreNom\.ssh\` sur Windows.
- Avant d'utiliser la clé privée, vous devez restreindre les permissions du fichier `.pem` pour qu'il ne soit lisible **que par vous**. Sinon, SSH refuse de l'utiliser (**WARNING: UNPROTECTED PRIVATE KEY FILE!**).

🖥 Pour Windows (PowerShell) :

```
$k = "$env:USERPROFILE\.ssh\votre-cle.pem"
icacls $k /inheritance:r                # supprime les droits hérités du dossier
icacls $k /grant:r "$($env:USERNAME):R" # seul votre compte peut lire la clé
icacls $k                             # vérification : une seule ligne doit
apparaître
```

🖥 Pour Linux et macOS :

```
chmod 400 ~/.ssh/votre-cle.pem
```

3.1.5. Tester la connexion depuis votre poste vers l'instance EC2

- Pour se connecter à votre instance EC2, vous aurez besoin de son adresse IP publique (ou de son nom DNS public). Vous trouverez cette information dans la console AWS, section "Instances".
- Cliquez ensuite sur "Se connecter" (*Connect*), onglet **SSH client** :

Se connecter Informations

Connectez-vous à une instance à l'aide du client basé sur un navigateur.

EC2 Instance Connect | Session Manager | **Client SSH** | EC2 Serial Console

ID d'instance
i-070e0b946133b4a23 (srvDevops)

1. Ouvrez un client SSH.
2. Recherchez votre fichier de clé privée. La clé utilisée pour lancer cette instance est 092025-ssh.pem
3. Exécuter, si nécessaire, cette commande pour vous assurer que votre clé n'est pas visible publiquement.
`chmod 400 "092025-ssh.pem"`
4. Connectez-vous à votre instance à l'aide de son DNS public :
`ec2-52-47-185-227.eu-west-3.compute.amazonaws.com`

Exemple :
`ssh -i "092025-ssh.pem" ubuntu@ec2-52-47-185-227.eu-west-3.compute.amazonaws.com`

📌 Remarque : Dans la plupart des cas, le nom d'utilisateur deviné est correct. Cependant, lisez les instructions d'utilisation de l'AMI pour vérifier si le propriétaire de l'AMI a modifié le nom d'utilisateur par défaut.

- Copiez la commande SSH fournie par AWS, qui ressemble à ceci :

```
ssh -i "votre-cle.pem" ubuntu@ec2-XX-XX-XX-XX.eu-west-3.compute.amazonaws.com
```

🖥 Ouvrez un terminal, placez-vous dans le répertoire où se trouve votre fichier `.pem` (ou indiquez son chemin complet après `-i`), puis exécutez la commande SSH.

- La première fois que vous vous connectez, vous recevrez un avertissement concernant l'authenticité de l'hôte. Tapez **yes** pour continuer.
- Si tout est configuré correctement, vous êtes maintenant connecté à votre instance EC2 : l'invite

de commande devient `ubuntu@ip-...:~$`.

```
PS C:\Users\baptiste\.ssh> ssh -i "092025-ssh.pem" ubuntu@ec2-52-47-185-227.eu-west-3.compute.amazonaws.com
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1011-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Sep  2 11:42:30 UTC 2025

System load:  0.0               Processes:    114
Usage of /:   36.8% of 6.71GB   Users logged in: 1
Memory usage: 27%              IPv4 address for enx0: 172.31.38.30
Swap usage:  0%

 * Ubuntu Pro delivers the most comprehensive open source security and
   compliance features.

https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

19 updates can be applied immediately.
17 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
Vous êtes connecté à votre instance
Last login: Tue Sep  2 08:08:42 2025 from 88.186.92.132
ubuntu@ip-172-31-38-30:~$
```

Se connecter avec une commande courte

Créez (ou complétez) le fichier `~/.ssh/config` de votre poste local :



```
Host jenkins
# remplacez XX.XX.XX.XX par votre Elastic IP
HostName XX.XX.XX.XX
User ubuntu
IdentityFile ~/.ssh/votre-cle.pem
```

Vous pourrez ensuite vous connecter simplement avec `ssh jenkins`.

Table 4. En cas de problème

Symptôme	Cause probable / solution
Connection timed out	Le port 22 n'est pas autorisé depuis votre IP dans le Security Group , ou votre IP a changé (réseau de l'établissement, partage de connexion...). Mettez à jour la règle My IP .
Permission denied (publickey)	Mauvaise clé (<code>-i</code>), mauvais utilisateur (c'est ubuntu pour une AMI Ubuntu), ou clé non associée à l'instance.
UNPROTECTED PRIVATE KEY FILE	Les permissions du fichier <code>.pem</code> sont trop ouvertes : reprenez la section précédente.
Host key verification failed	L'instance a été recréeée avec la même IP. Supprimez l'ancienne empreinte : <code>ssh-keygen -R XX.XX.XX.XX</code> .

3.1.6. Installer Docker Engine sur l'instance

Jenkins et l'application seront exécutés dans des conteneurs : Docker doit donc être installé **sur l'instance EC2**.

-  **Instance EC2** : installez Docker Engine et le plugin Compose depuis le dépôt officiel Docker :

```
# Dépendances et clé GPG du dépôt Docker
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Ajout du dépôt Docker
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(.
/etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null

# Installation
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

-  Autorisez l'utilisateur **ubuntu** à utiliser Docker sans **sudo**, puis **déconnectez-vous et reconnectez-vous** en SSH pour que le changement de groupe soit pris en compte :

```
sudo usermod -aG docker ubuntu
exit
```

-  Après reconnexion, vérifiez l'installation :

```
docker version
docker compose version
docker run --rm hello-world
```

✔ **Vous devriez voir** : les versions du client et du serveur Docker, la version de Docker Compose, puis le message **Hello from Docker!**.

3.1.7. Vérifier l'étape

 **Instance EC2** : clonez votre dépôt (il est public) et lancez la vérification de l'étape 7 :

```
git clone https://github.com/<votre_compte_github>/jenkins-calculatrice.git
cd jenkins-calculatrice
```

```
./check.sh 7
```

Le script vérifie le système, Docker (utilisable sans `sudo`), Docker Compose, la mémoire et l'espace disque, et affiche l'adresse IP publique de l'instance : elle doit correspondre à votre Elastic IP.

3.1.8. Arrêter l'instance en fin de séance

Une instance EC2 est facturée tant qu'elle est **en cours d'exécution** (*running*).

- À la fin de chaque séance : **EC2** → **Instances** → sélectionnez l'instance → **Instance state** → **Stop instance**.
- À la fin de la séquence : **Terminate instance**, puis **libérez l'Elastic IP** (**Elastic IPs** → **Release**). Une Elastic IP est facturée tant qu'elle n'est pas libérée, même si l'instance est arrêtée ou supprimée.



Les conteneurs configurés avec `restart: unless-stopped` (Jenkins et la calculatrice, au chapitre suivant) redémarrent automatiquement quand vous relancez l'instance.

3.2. Récapitulatif

- ✔ Une paire de clés SSH, protégée sur votre poste.
- ✔ Une instance EC2 Ubuntu avec une Elastic IP et un Security Group limité à votre IP.
- ✔ Une connexion SSH fonctionnelle.
- ✔ Docker Engine et Docker Compose installés sur l'instance.
- ✔ `./check.sh 7` validé.

Vous êtes prêt pour installer Jenkins sur le serveur.

4. Jenkins sur AWS et webhook GitHub

Version 2.0.0 | Dernière mise à jour : 28/09/2026

Durée de réalisation : 3 heures

Auteur : Bauer Baptiste

4.1. Étape 8 : Jenkins sur AWS et webhook GitHub

🎯 **Objectif** : faire tourner la chaîne CI/CD sur le serveur AWS, et faire en sorte que **GitHub prévienne Jenkins** à chaque push, au lieu que Jenkins aille vérifier toutes les 2 minutes.

Prérequis

- Étape 7 validée : `./check.sh 7` sur l'instance EC2.
- Votre dépôt contient le **Jenkinsfile** complet de l'étape 6.

💡 Comprendre

Un **webhook** est un mécanisme qui permet à GitHub de prévenir automatiquement une application externe (ici Jenkins) lorsqu'un événement survient dans un dépôt (un push, une pull request, une release...). GitHub envoie alors une requête HTTP (un message JSON) à l'URL indiquée.

	Scrutation (étapes 3 à 6)	Webhook (cette étape)
Qui prend l'initiative ?	Jenkins interroge GitHub régulièrement.	GitHub prévient Jenkins à chaque push.
Délai avant le build	Jusqu'à 2 minutes.	Quelques secondes.
Requêtes inutiles	720 par jour, même sans commit.	Aucune.
Condition	Jenkins doit pouvoir joindre GitHub.	GitHub doit pouvoir joindre Jenkins depuis Internet.

C'est cette dernière condition qui impose un serveur accessible depuis Internet : votre poste ne l'était pas, l'instance EC2 l'est.

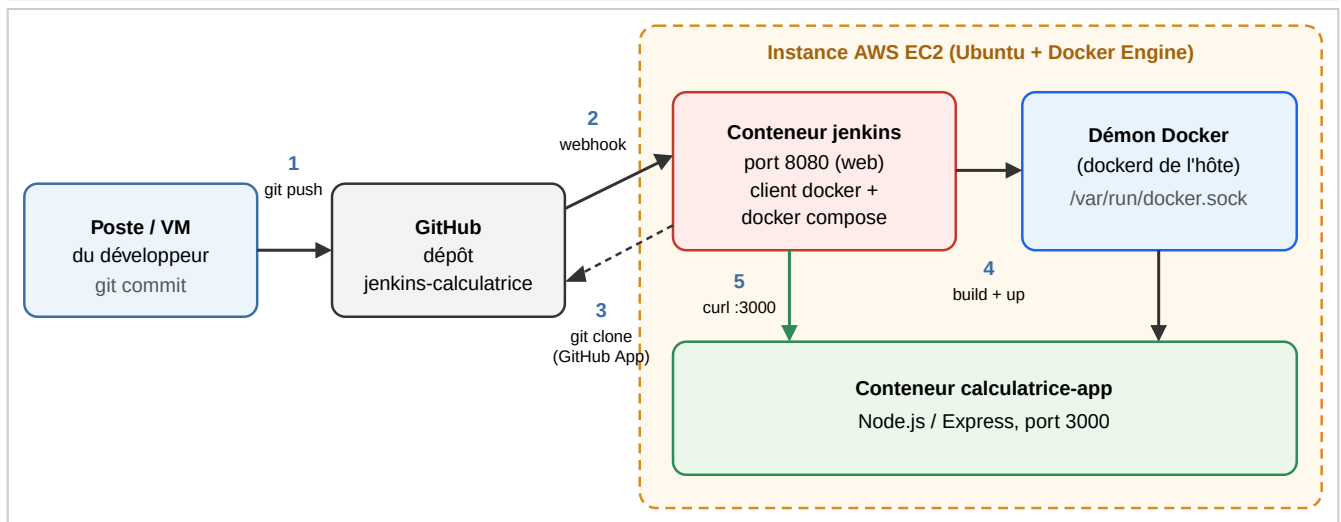


Figure 4. Architecture cible

4.1.1. 8.1 Lancer Jenkins sur le serveur

🔧 À faire

1. **Instance EC2** : dans le clone de votre dépôt (étape 7), récupérez la dernière version puis préparez le fichier `.env` :

```
cd ~/jenkins-calculatrice
git pull
cd jenkins
./init-env.sh
```

Cette fois, pour l'URL de Jenkins, indiquez l'adresse publique du serveur : `http://<ip_public>:8080/` (votre Elastic IP).

2. **Lancez Jenkins** :

```
docker compose up -d --build
```

3. Dans la console AWS, ajoutez au **Security Group** de l'instance une règle entrante **Custom TCP**, port **8080**, source **My IP**.

4. Ouvrez http://<ip_public>:8080 dans votre navigateur et connectez-vous.



- N'ouvrez pas le port **8080** à tout Internet (**0.0.0.0/0**) : seuls votre IP et GitHub en ont besoin.
- Jenkins est ici servi en **HTTP** : votre mot de passe circule en clair. En production, on place Jenkins derrière un serveur web en **HTTPS** (Nginx, Caddy, Traefik...).

4.1.2. 8.2 Recréer le pipeline

Ce Jenkins est tout neuf : il ne connaît pas encore votre pipeline. Grâce au **Jenkinsfile**, un seul job suffit à tout recréer.

🔧 À faire

1. Créez un job de type **Pipeline** nommé **CalculatricePipeline** :
 - **Definition** : Pipeline script from SCM ;
 - **SCM** : Git, **Repository URL** : https://github.com/<votre_compte_github>/jenkins-calculatrice.git ;
 - **Branch Specifier** : /main, *Script Path : Jenkinsfile.
2. Lancez un build : l'application est construite, testée et déployée **sur le serveur**.
3. Ajoutez au Security Group une règle entrante pour le port **3000** (source **My IP**), puis ouvrez http://<ip_public>:3000.



Remarquez l'intérêt du **Jenkinsfile** : toute la chaîne (tests, build, déploiement, smoke test, déclencheur) est décrite dans le dépôt. Recréer le pipeline sur un nouveau serveur ne demande que quelques clics.

4.1.3. 8.3 Autoriser GitHub à joindre Jenkins

Les notifications de GitHub arrivent depuis ses propres serveurs : elles sont bloquées tant que le port **8080** n'est ouvert qu'à votre IP.

🔧 À faire

Dans le Security Group, ajoutez des règles entrantes **Custom TCP**, port **8080**, pour les plages d'adresses utilisées par les webhooks GitHub. Elles sont listées dans la rubrique "hooks" de <https://api.github.com/meta>, par exemple :

```
192.30.252.0/22
185.199.108.0/22
140.82.112.0/20
143.55.64.0/20
```

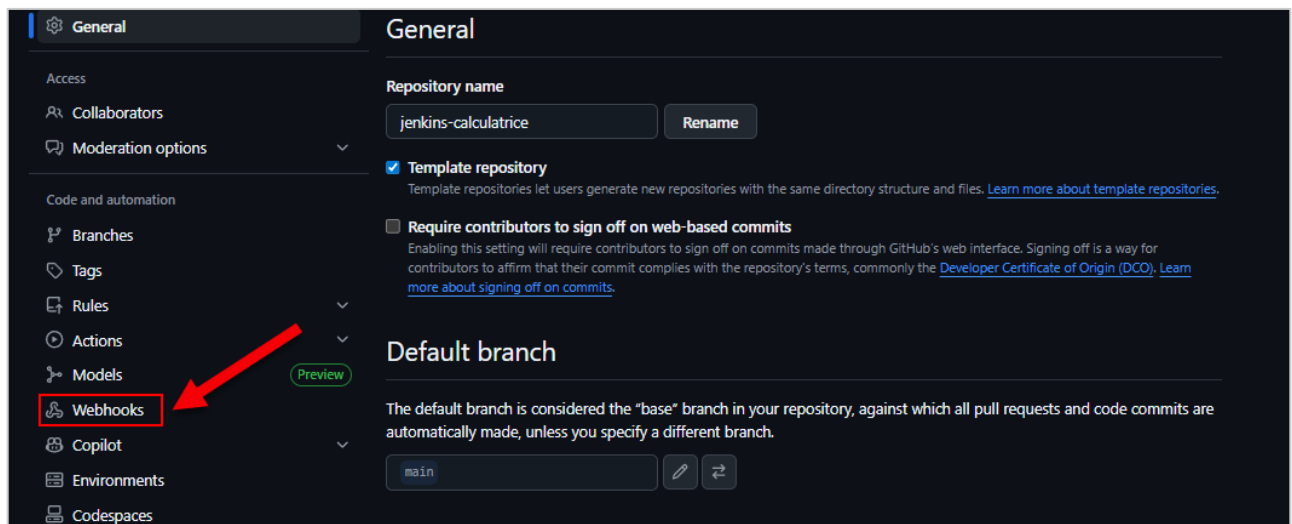


Ces plages peuvent évoluer : en cas de webhook sans réponse (*timeout*), comparez vos règles avec la liste à jour.

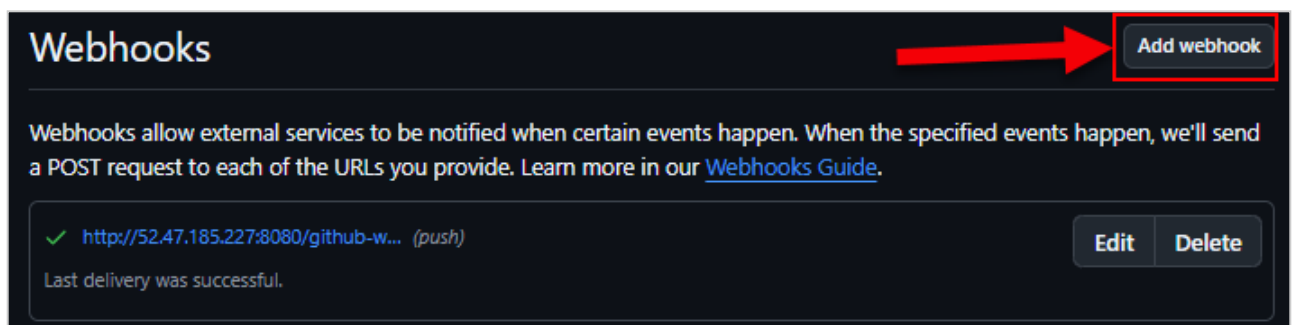
4.1.4. 8.4 Créer le webhook

🔧 À faire

1. Sur GitHub, ouvrez les **Settings** de votre dépôt, puis **Webhooks** :



2. Cliquez sur **Add webhook** :



3. Remplissez le formulaire :

- **Payload URL** : `http://<ip_public>:8080/github-webhook/` (le / final est obligatoire) ;
- **Content type** : `application/json` ;
- **Secret** : une phrase secrète de votre choix (notez-la, elle servira en 8.6) ;

- **Which events** : *Just the push event.*

The screenshot shows the 'Manage webhook' configuration page in Jenkins. It has two tabs: 'Settings' (selected) and 'Recent Deliveries'. The 'Settings' tab contains the following fields and options:

- Payload URL ***: A text input field containing 'http://52.47.185.227:8080/github-webhook/'. A red arrow points to this field with the text 'Adresse IP de votre server avec Jenkins et le port utilisé par Jenkins'.
- Content type ***: A dropdown menu with 'application/json' selected. A red box highlights this field.
- Secret**: A section with an information icon and text: 'There is currently a secret configured for this webhook. If you've lost or forgotten this secret, you can change it, but be aware that any integrations using this secret will need to be updated.' Below this is a 'Change secret' button. A red arrow points to this section with the text 'Saisissez une phrase secrète'.
- SSL verification**: A section with a lock icon and text: 'By default, we verify SSL certificates when delivering payloads.' Below this are two radio buttons: 'Enable SSL verification' (selected) and 'Disable (not recommended)'.
- Which events would you like to trigger this webhook?**: A section with three radio buttons: 'Just the push event.' (selected and highlighted with a red box), 'Send me everything.', and 'Let me select individual events.'
- Active**: A checkbox labeled 'Active' which is checked. Below it is the text: 'We will deliver event details when this hook is triggered.'
- At the bottom are two buttons: 'Update webhook' (green) and 'Delete webhook' (grey).

4. Validez. GitHub envoie aussitôt un événement de test (*ping*). Dans l'onglet **Recent Deliveries** du webhook, il doit apparaître avec une coche verte et un code de réponse **200**.

4.1.5. 8.5 Déclencher le pipeline par le webhook

💡 Le déclencheur est écrit dans le **Jenkinsfile** : il suffit de remplacer la scrutation par l'écoute du webhook.

🔧 À faire

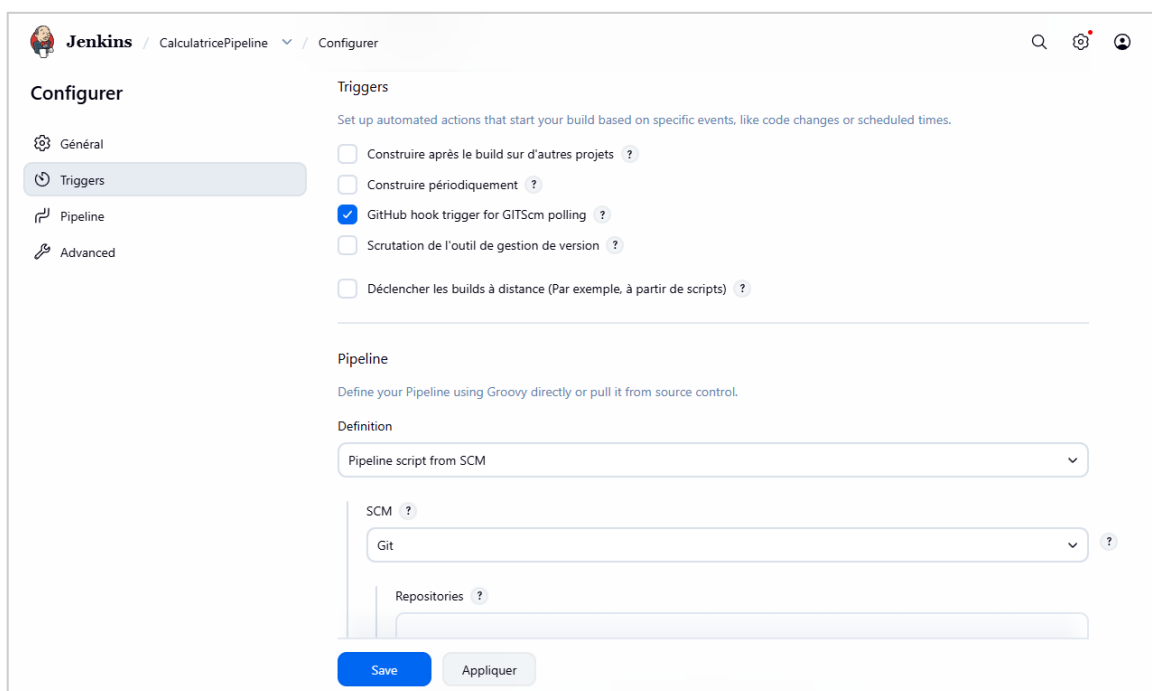
1. 🖥 **Sur votre poste**, dans le **Jenkinsfile**, remplacez le bloc **triggers** :

```
triggers {  
    // Lancé par le webhook GitHub à chaque push  
    githubPush()  
}
```

puis poussez :

```
git commit -am "Pipeline déclenché par le webhook GitHub"  
git push
```

2. Dans Jenkins (sur le serveur), lancez **une fois à la main** **CalculatricePipeline** : ce build enregistre le nouveau déclencheur. Dans **Configure** → **Triggers**, la case **GitHub hook trigger for GITScm polling** est maintenant cochée.



3. 🖥️ Faites une modification visible, par exemple le numéro de version dans le titre principal de **public/index.html** (la balise **<h1>**, « Calculatrice d'Addition VERSION 2.1 »), puis :

```
git commit -am "Nouvelle version de la page"  
git push
```

✓ Vérifier

- Dans GitHub → **Settings** → **Webhooks** → **Recent Deliveries** : une livraison **push** avec le code **200**.
- Dans Jenkins : un build démarre **dans les secondes qui suivent** le push. Sa page indique « **Started by GitHub push by <votre_compte_github>** ».
- Sur http://<ip_public>:3000 : le nouveau titre.

Webhooks

Add webhook

Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our [Webhooks Guide](#).

✓ http://35.180.46.161:8080/github-w... (push)
Edit Delete

Last delivery was successful.

☁ Instance EC2 :

```
cd ~/jenkins-calculatrice
git pull
./check.sh 8
```



Bloqué ? La branche `solution-etape-8` contient le `Jenkinsfile` avec `githubPush()`.

4.1.6. 8.6 Faire vérifier le secret par Jenkins (approfondissement)

Le secret du webhook permet à Jenkins de s'assurer qu'une requête vient réellement de GitHub : GitHub signe chaque message avec ce secret, et Jenkins vérifie la signature.

1. Dans **Administrer Jenkins** → **Credentials** → **System** → **Identifiants globaux** → **Add Credentials**, créez un credential de type **Secret text** contenant la phrase secrète du webhook.
2. Dans **Administrer Jenkins** → **System**, section **GitHub** → **Avancé** → **Shared secrets**, sélectionnez ce credential, puis sauvez.
3. Poussez un commit : le build doit toujours se déclencher. Modifiez volontairement le secret côté GitHub : les livraisons sont alors refusées par Jenkins.

✖ En cas de problème

Symptôme	Cause probable / solution
Recent Deliveries : <code>We couldn't deliver this payload: timed out</code>	Le Security Group bloque GitHub sur le port <code>8080</code> (étape 8.3), ou Jenkins est arrêté.
Recent Deliveries : code <code>302</code> ou <code>404</code>	URL incorrecte : elle doit se terminer par <code>/github-webhook/</code> , avec le <code>/</code> final.
Livraison <code>200</code> mais aucun build	Le pipeline n'a pas été lancé une fois à la main après l'ajout de <code>githubPush()</code> , ou l'URL du dépôt dans le job ne correspond pas à celle du dépôt qui envoie le webhook.
Deux builds par push	Deux webhooks pointent vers Jenkins (par exemple celui d'une GitHub App, au chapitre suivant). N'en gardez qu'un.
<code>check.sh</code> : <code>JENKINS_URL</code> vaut <code>http://localhost:8080/</code>	Relancez <code>./init-env.sh</code> avec l'adresse publique, puis <code>docker compose up -d</code> .

Q13) Pourquoi faut-il lancer une fois le pipeline à la main après avoir ajouté `githubPush()` dans le `Jenkinsfile` ?

réponse 13 disponible.

Q14) Un camarade propose d'ouvrir le port `8080` à `0.0.0.0/0` « pour que le webhook marche à coup sûr ». Quels risques cela présente-t-il, compte tenu de la façon dont Jenkins est installé ?

réponse 14 disponible.

4.2. Récapitulatif

- ✔ Jenkins et l'application tournent sur un serveur accessible depuis Internet.
- ✔ Le pipeline a été recréé en quelques clics grâce au `Jenkinsfile`.
- ✔ GitHub prévient Jenkins à chaque push : le build démarre en quelques secondes.
- ✔ Le port `8080` n'est ouvert qu'à votre IP et à GitHub.



N'oubliez pas d'**arrêter l'instance** en fin de séance (voir le chapitre *Préparer une instance AWS EC2*). En fin de séquence : `docker compose down -v` dans `jenkins/`, suppression du webhook, puis résiliation de l'instance et libération de l'Elastic IP.

5. Dépôt privé et GitHub App

Version 2.0.0 | Dernière mise à jour : 28/09/2026

Durée de réalisation : 2 heures

Auteur : Bauer Baptiste

5.1. Étape 9 : Dépôt privé et GitHub App (approfondissement)

🎯 **Objectif** : que Jenkins puisse cloner un dépôt **privé**, avec des identifiants dédiés et limités, plutôt qu'avec votre compte personnel.

Prérequis

- Étape 8 validée : `./check.sh 8` sur l'instance EC2.

💡 Comprendre

Jusqu'ici, votre dépôt était public : Jenkins pouvait le cloner sans s'authentifier. En entreprise, le code est presque toujours **privé**. Jenkins a alors besoin d'identifiants. Plusieurs solutions existent :

Solution	Avantage	Inconvénient
Votre mot de passe ou un token personnel	Simple.	Jenkins agit en votre nom , avec tous vos droits, sur tous vos dépôts. Si vous quittez l'équipe, la CI s'arrête.
Une clé SSH de déploiement	Limitée à un dépôt.	Une clé par dépôt, pas d'accès à l'API GitHub (statuts de build...).
Une GitHub App	Identité « machine » propre, permissions fines, installable sur plusieurs dépôts, jetons de courte durée.	Un peu plus longue à configurer.

Une **GitHub App** est une application déclarée dans GitHub. Elle s'authentifie avec un **App ID** et une **clé privée**, et n'a accès qu'aux dépôts sur lesquels vous l'installez, avec les permissions que vous choisissez.

5.1.1. 9.1 Constater le problème

🔧 À faire

- Sur GitHub, rendez votre dépôt privé : **Settings** → **General** → **Danger Zone** → **Change visibility** → **Make private**.
- Dans Jenkins, lancez `CalculatricePipeline`. Le build échoue dès la récupération du code :

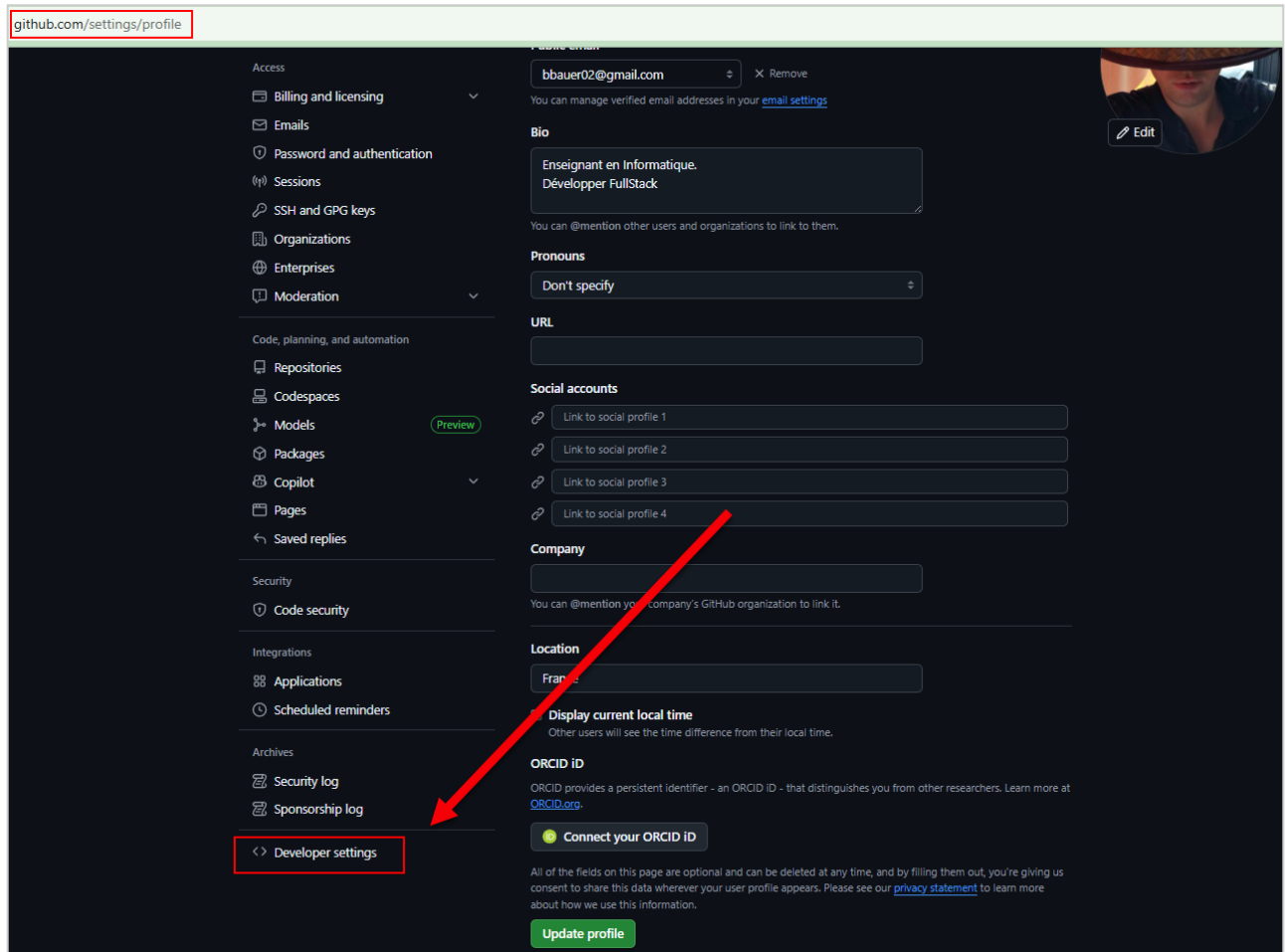
```
ERROR: Error fetching remote repo 'origin'
... Authentication failed / repository not found
```

GitHub refuse de donner le code d'un dépôt privé à un visiteur anonyme.

5.1.2. 9.2 Créer la GitHub App

🔧 À faire

1. Sur GitHub : cliquez sur votre photo de profil → **Settings** → **Developer settings** → **GitHub Apps** → **New GitHub App**.



2. Remplissez le formulaire :
 - **GitHub App name** : un nom de votre choix (il doit être unique sur GitHub, ex. `jenkins-<votre_compte_github>`);
 - **Homepage URL** : l'URL de votre Jenkins, `http://<ip_public>:8080/`.

The screenshot shows the 'Register new GitHub App' form. A red arrow points to the 'GitHub App name' field containing 'calculatrice-jenkins', with the text 'Un nom arbitraire' next to it. Another red arrow points to the 'Homepage URL' field containing 'http://52.47.185.227:8080/', with the text 'URL de Jenkins' next to it. The form includes sections for 'Identifying and authorizing users' and 'Callback URL'.

Register new GitHub App

GitHub App name *

calculatrice-jenkins

The name of your GitHub App.

Write **Preview** Markdown supported

This is displayed to users of your GitHub App

Homepage URL *

http://52.47.185.227:8080/

The full URL to your GitHub App's website.

Identifying and authorizing users **Add Callback URL**

The full URL to redirect to after a user authorizes an installation.
Read our [Callback URL documentation](#) for more information.

Callback URL

e.g. https://example.com/auth **Delete**

☒ **Expire user authorization tokens**

This will provide a refresh_token which can be used to request an updated access token when this access token expires.

☐ **Request user authorization (OAuth) during installation**

Requests that the installing user grants access to their identity during installation of your App
Read our [Identifying and authorizing users for GitHub Apps documentation](#) for more information.

☐ **Enable Device Flow**

Allow this GitHub App to authorize users via the Device Flow.
Read the [Device Flow documentation](#) for more information.

3. Dans la section **Webhook**, décochez la case **Active**.



Le webhook du dépôt (étape 8) prévient déjà Jenkins. Si le webhook de l'App restait actif avec la même URL, chaque push serait envoyé **deux fois** à Jenkins. L'App ne sert ici qu'à l'**authentification**.

4. Dans **Repository permissions**, donnez l'accès en lecture au contenu du dépôt (**Contents** : *Read-only*) ; les autres permissions restent à leur valeur par défaut :

The screenshot shows the 'Repository permissions' section. A red arrow points to the 'Contents' dropdown menu, which is set to 'Selected'. Another red box highlights the 'Access: Read-only' dropdown menu.

Contents **Selected** **Access: Read-only**

Repository contents, commits, branches, downloads, releases, and merges. [Learn more.](#)

Metadata **Mandatory**
Search repositories, list collaborators, and access repository metadata. [Learn more.](#) Access: Read-only ▾

Pull requests **Selected**
Pull requests and related comments, assignees, labels, milestones, and merges. [Learn more.](#) Access: Read-only ▾

5. Dans **Where can this GitHub App be installed?**, choisissez **Only on this account**, puis cliquez sur **Create GitHub App**.

Where can this GitHub App be installed?

☒ **Only on this account**
Only allow this GitHub App to be installed on the @bbauer02 account.

☐ **Any account**
Allow this GitHub App to be installed by any user or organization.

Create GitHub App **Cancel**

Registration successful. You must [generate a private key](#) in order to install your GitHub App.

Settings / Developer settings / GitHub Apps / calculatrice-jenkins2

About

Owned by: @bbauer02
App ID: 1888807
Using your App ID to get installation tokens? You can now [use your Client ID instead.](#)
Client ID: lv23lizJd2CxyYm7TAj
Revoke all user tokens

GitHub Apps can use OAuth credentials to identify users. Learn more about identifying users by reading our [integration developer documentation](#).

Client secrets **Generate a new client secret**

You need a client secret to authenticate as the application to the API.

Basic information

GitHub App name *
calculatrice-jenkins2
The name of your GitHub App.

Cliquez et téléchargez la clé .PEM

6. Sur la page de l'App, notez son App ID.

5.1.3. 9.3 Installer l'App sur votre dépôt

1. Sur la page de votre GitHub App, cliquez sur **Install App** dans le menu de gauche, puis sur **Install** à côté de votre compte.
2. Choisissez **Only select repositories**, sélectionnez **jenkins-calculatrice** et validez.

5.1.4. 9.4 Générer et convertir la clé privée

1. Sur la page de l'App, section **Private keys**, cliquez sur **Generate a private key** : un fichier **.pem** est téléchargé sur votre poste.
2. GitHub fournit la clé au format **PKCS#1** (**BEGIN RSA PRIVATE KEY**), alors que Jenkins exige le format **PKCS#8** (**BEGIN PRIVATE KEY**). ☁ Sur l'instance EC2, créez un fichier temporaire et collez-y le contenu du **.pem** :

```
nano ~/github-app-pkcs1.pem
```

3. ☁ Convertissez la clé, puis affichez le résultat :

```
openssl pkcs8 -topk8 -inform PEM -outform PEM -nocrypt \
  -in ~/github-app-pkcs1.pem -out ~/github-app-pkcs8.pem
cat ~/github-app-pkcs8.pem
```

✓ Le résultat commence par **-----BEGIN PRIVATE KEY-----**. Copiez l'intégralité du contenu.

5.1.5. 9.5 Enregistrer l'identifiant dans Jenkins

1. Dans Jenkins : **Administrer Jenkins** → **Credentials** → **System** → **Identifiants globaux (illimité)** → **Add Credentials**.

The screenshot shows the Jenkins 'Administrer Jenkins' interface. The navigation path is highlighted with red arrows: **System** (under Configuration du système) → **Identifiants globaux (illimité)** (under Sécurité) → **+ Add Credentials** (button).

Configuration du système

- System**: Configurer les paramètres généraux et les chemins de fichiers.
- Tools**: Configurer les outils, leur localisation et les installeurs automatiques.
- Plugins**: Ajouter, supprimer, activer ou désactiver des plugins qui peuvent étendre les fonctionnalités de Jenkins.
- Nodes**: Ajouter, supprimer, contrôler et monitorer les divers nœuds que Jenkins utilise pour exécuter les jobs.
- Clouds**: Ajouter, supprimer et configurer les instances de cloud afin de provisionner les agents à la demande.
- Apparence générale**: Configurer l'apparence générale de Jenkins.

Sécurité

- Security**: Sécuriser Jenkins; définir qui est autorisé à accéder au système.
- Credentials**: Configurer credentials.
- Credential Providers**: Configure the credential providers and types.
- Users**: Créer/supprimer/modifier les utilisateurs qui peuvent se logger sur ce serveur Jenkins.

Identifiants globaux (illimité)

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Nom	Type	Description
github-app-jenkins	1887377/***** (github-app-jenkins)	GitHub App	github-app-jenkins

2. Choisissez le type **GitHub App**, puis renseignez :

- **ID** : `calculatrice-auth` ;
- **Description** : `Authentification via GitHub App` ;
- **App ID** : l'App ID noté précédemment ;
- **Key** : la clé au format PKCS#8 ;
- dans **Avancé** : **Owner** = `<votre_compte_github>`, **Repositories** = `jenkins-calculatrice`.

Update credentials

ID ?
calculatrice-auth

Description ?
Authentification via GitHub App

App ID ?
1898308

Key ?
Caché pour des raisons de confidentialité

Avancé ^

Repository access strategy ?
Specify accessible repositories

Owner ?
bbauer02

Repositories ?
jenkins-calculatrice

Default permissions strategy ?
All permissions available to the app installation

3. Cliquez sur **Test Connection** : un message de succès indique le nombre de dépôts accessibles.

Test Connection

GitHub organization to test against
bbauer02

Success, Remaining rate limit: 5000

Test Connection

4. ☁ Supprimez les fichiers temporaires, et le `.pem` téléchargé sur votre poste (ou rangez-le dans un gestionnaire de mots de passe) :

```
rm ~/github-app-pkcs1.pem ~/github-app-pkcs8.pem
```



Une clé privée ne doit **jamais** être déposée dans un dépôt Git, envoyée par messagerie ou laissée dans un dossier partagé. En cas de fuite, supprimez-la immédiatement dans la page de l'App (**Private keys** → **Delete**).

5.1.6. 9.6 Utiliser l'identifiant dans le pipeline

1. Dans `CalculatricePipeline` → **Configure** → **Pipeline** → **SCM**, sélectionnez le credential `calculatrice-auth` dans la liste **Credentials** (la capture ci-dessous montre le même champ dans

un job free-style) :

2. Sauvez, puis lancez un build : le dépôt privé est cloné, le pipeline est vert.
3. Poussiez une modification : le webhook déclenche toujours le pipeline.

✓ Vérifier

☁ Instance EC2 :

```
cd ~/jenkins-calculatrice
./check.sh 9
```



Le dépôt étant maintenant privé, un `git pull` sur l'instance vous demandera de vous authentifier. Le plus simple est d'ajouter une clé SSH de l'instance à votre compte GitHub (`ssh-keygen -t ed25519`, puis ajout du contenu de `~/.ssh/id_ed25519.pub` sur <https://github.com/settings/keys>), et de passer le clone en SSH : `git remote set-url origin git@github.com:<votre_compte_github>/jenkins-calculatrice.git`.

⊗ En cas de problème

Symptôme	Cause probable / solution
Test Connection : Failed to decode private key ou erreur de format	La clé n'est pas au format PKCS#8 : elle doit commencer par BEGIN PRIVATE KEY , sans RSA .
Test Connection : Could not find installation	L'App n'est pas installée sur votre compte ou sur ce dépôt (9.3), ou le champ Owner est mal saisi.

Symptôme	Cause probable / solution
Le build échoue toujours à la récupération du code	Le credential n'est pas sélectionné dans la configuration SCM du pipeline, ou l'App n'a pas la permission Contents .
Deux builds par push	Le webhook de la GitHub App est resté actif : décochez Active dans la section Webhook de l'App.

Q15) Vous quittez l'équipe et votre compte GitHub est retiré de l'organisation. Que devient la CI si Jenkins utilisait votre token personnel ? Et s'il utilise une GitHub App ?

réponse 15 disponible.

5.2. Pour aller plus loin

- **Multibranch Pipeline** : un pipeline par branche et par pull request, créé automatiquement pour chaque branche qui contient un **Jenkinsfile**. En donnant à la GitHub App la permission **Checks** (*Read and write*), le résultat de chaque build s'affiche directement dans la pull request, et on peut interdire la fusion d'une PR dont le build est rouge.
- **Publication de l'image** sur un registre (Docker Hub, GitHub Container Registry), avec un credential Jenkins : la même image testée peut alors être déployée sur plusieurs serveurs.
- **Retour arrière** (*rollback*) : grâce au tag par commit (**calculatorice:<hash>**), relancer une version précédente en une commande.

5.3. Récapitulatif

- ✔ Le dépôt est privé.
- ✔ Jenkins s'authentifie auprès de GitHub avec une GitHub App, limitée à ce dépôt, en lecture seule.
- ✔ La clé privée n'est plus stockée que dans Jenkins (et, éventuellement, dans votre gestionnaire de mots de passe).

6. Mini-projet : votre propre chaîne CI/CD

Version 2.0.0 | Dernière mise à jour : 28/09/2026

Durée de réalisation : 6 heures

Auteur : Bauer Baptiste

6.1. Étape 10 : Mini-projet en autonomie (défi)

🎯 **Objectif** : mettre en place, **sans guide pas à pas**, une chaîne d'intégration continue complète pour une autre application que la calculatrice.

Ce mini-projet vérifie que vous savez **transposer** ce que vous avez appris : les commandes ne sont plus données, à vous de retrouver dans les chapitres précédents (et dans le mémo) ce dont vous avez besoin.

6.1.1. Choisir une application

Choisissez **une** application parmi :

- une application que vous avez développée dans un autre cours ou en projet personnel (Node.js, PHP, Python, Java...) ;
- l'application Laravel du TD *Projet Laravel complexe et Docker* du cours Docker ;
- une petite API que vous écrivez pour l'occasion (par exemple une API de conversion d'unités), avec au moins trois tests automatisés.



L'application doit posséder des **tests automatisés** qui se lancent en une commande (`npm test`, `php artisan test`, `pytest`, `mvn test`...). Si elle n'en a pas, écrivez-en au moins trois : c'est une partie du travail.

6.1.2. Cahier des charges

#	Exigence
E1	Le code est dans un dépôt GitHub dont vous êtes propriétaire.
E2	Un Dockerfile multi-stage : un stage exécute les tests, l'image finale ne contient que le nécessaire à l'exécution et tourne avec un utilisateur non-root.
E3	Un fichier Compose (nom de projet fixé) lance l'application, et au besoin ses services (base de données...).
E4	Un Jenkinsfile versionné contient au moins les stages Tests , Build , Déploiement et Smoke test , et affiche les logs de l'application en cas d'échec (bloc post).
E5	Le pipeline se déclenche automatiquement à chaque push (scrutation en local, ou webhook sur le serveur).
E6	Vous avez démontré qu'un test en échec bloque le déploiement : un build rouge, suivi d'un build vert, apparaissent dans l'historique.

#	Exigence
E7	Un README.md explique en quelques lignes comment lancer l'application et à quoi sert chaque fichier de la chaîne CI/CD.

6.1.3. Livrables

1. L'URL de votre dépôt GitHub (public, ou partagé avec votre enseignant).
2. Une capture de la vue des stages du pipeline, avec au moins un build rouge et un build vert.
3. Une capture de la sortie console du build rouge, sur laquelle vous avez entouré la cause de l'échec.
4. Un court texte (10 lignes maximum) : les difficultés rencontrées et comment vous les avez résolues.

6.1.4. Grille d'auto-évaluation

Évaluez votre travail avant de le rendre. Votre enseignant utilisera la même grille.

Critère	Points	Auto-évaluation
E2 — Le Dockerfile exécute les tests pendant la construction	3	
E2 — Image finale sans outils de développement, utilisateur non-root	2	
E3 — L'application se lance avec docker compose up -d --build	2	
E4 — Jenkinsfile avec 4 stages cohérents, pipeline vert	4	
E4 — Bloc post utile en cas d'échec	1	
E5 — Déclenchement automatique à chaque push	2	
E6 — Démonstration qu'un test en échec bloque le déploiement	3	
E7 — README.md clair et exact	1	
Livrable 4 — Analyse des difficultés	2	
Total	20	

6.1.5. Questions pour la soutenance

Préparez-vous à répondre oralement à ces questions sur **votre** projet :

1. Montrez dans votre **Dockerfile** à quel moment les tests sont exécutés. Que se passe-t-il pour l'application en ligne si un test échoue ?
2. Pourquoi votre **Jenkinsfile** est-il dans le dépôt plutôt que saisi dans l'interface de Jenkins ?
3. Quelle est la différence entre vos tests automatisés et votre smoke test ?
4. Votre Jenkins a accès au socket Docker : quel risque cela représente-t-il, et comment le limiter ?
5. Si vous deviez déployer la même image sur trois serveurs, que faudrait-il ajouter à votre chaîne

?



Bloqué ? Relisez l'étape correspondante du cours, et utilisez le mémo du dernier chapitre. Les rubriques 🚫 *En cas de problème* couvrent la plupart des erreurs courantes.

7. Mémo et glossaire

Version 2.0.0 | Dernière mise à jour : 28/09/2026

Auteur : Bauer Baptiste

7.1. Mémo des commandes

7.1.1. Outils du TP

Commande	Rôle
<code>./check.sh <étape></code>	Vérifie l'étape indiquée (à lancer à la racine du dépôt).
<code>jenkins/init-env.sh</code>	Crée <code>jenkins/.env</code> : identifiants admin, port, URL, groupe du socket Docker.
<code>git fetch https://github.com/bbauer02/jenkins-calculatrice.git solution-etape-N</code> <code>git checkout FETCH_HEAD~1.</code>	Récupère les fichiers de correction de l'étape N.

7.1.2. Jenkins (dans le dossier jenkins/)

Commande	Rôle
<code>docker compose up -d --build</code>	Construit l'image et démarre Jenkins.
<code>docker compose ps</code>	État du conteneur.
<code>docker logs -f jenkins</code>	Journal de Jenkins en direct (Ctrl+C pour quitter).
<code>docker compose restart</code>	Redémarre Jenkins.
<code>docker compose down</code>	Supprime le conteneur ; les données (volume <code>jenkins_home</code>) sont conservées.
<code>docker compose down -v</code>	Supprime le conteneur et toutes les données de Jenkins.
<code>docker exec jenkins docker version</code>	Vérifie que Jenkins peut piloter Docker.

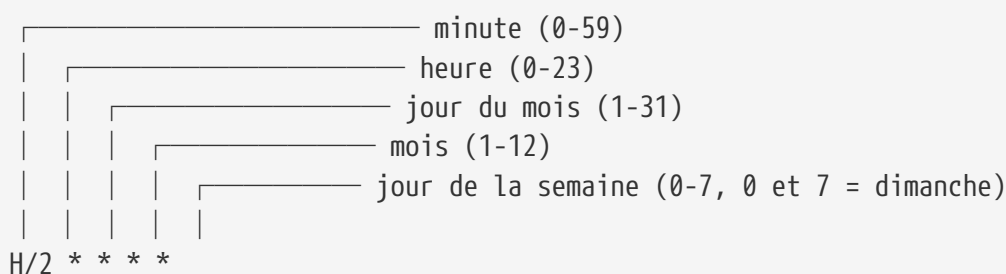
7.1.3. Application (à la racine du dépôt)

Commande	Rôle
<code>docker compose up -d --build</code>	Construit l'image (tests compris) et (re)lance l'application.
<code>docker compose logs --tail=50</code>	Dernières lignes du journal de l'application.
<code>docker compose down</code>	Arrête et supprime le conteneur de l'application.
<code>docker build --target builder -t calculatrice:test .</code>	Construit uniquement le stage <code>builder</code> (exécute les tests).
<code>curl -fsS http://localhost:3000</code>	Vérifie que l'application répond.

7.1.4. Git

Commande	Rôle
<code>git status</code>	Fichiers modifiés, et commits non poussés.
<code>git add <fichier></code> puis <code>git commit -m "message"</code>	Enregistre une modification.
<code>git commit -am "message"</code>	Enregistre toutes les modifications de fichiers déjà suivis.
<code>git push</code>	Envoie les commits sur GitHub (Jenkins ne voit que ce qui est sur GitHub).
<code>git revert --no-edit HEAD</code>	Crée un commit qui annule le dernier commit.
<code>git log --oneline -5</code>	Les 5 derniers commits.

7.1.5. Syntaxe cron (scrutation)



Planning	Signification
<code>H/2 * * * *</code>	Toutes les 2 minutes environ.
<code>H/15 * * * *</code>	Toutes les 15 minutes environ.
<code>H 2 * * *</code>	Une fois par nuit, vers 2 h.
<code>H 8 * * 1-5</code>	Vers 8 h, du lundi au vendredi.

Le **H** (*hash*) choisit une valeur fixe mais propre à chaque job, pour éviter que tous les jobs démarrent au même instant.

7.1.6. Anatomie d'un Jenkinsfile déclaratif

```

pipeline {
  agent any                                // où exécuter
  triggers { githubPush() }                // quand démarrer
  options { disableConcurrentBuilds() }    // réglages
  environment { APP_URL = 'http://...' }  // variables
  stages {
    stage('Tests') {
      steps { sh 'commande' }              // une ou plusieurs étapes
    }
  }
}

```

```

post {
    success { echo 'OK' } // selon le résultat
    failure { sh 'docker compose logs --tail=50 || true' }
    always { sh 'docker image prune -f || true' }
}
}

```

7.2. Glossaire

Terme	Définition
Agent	Machine (ou conteneur) sur laquelle Jenkins exécute des builds. Dans ce cours, Jenkins exécute tout lui-même (contrôleur).
Build	Une exécution d'un job, numérotée (#1 , #2...), avec sa sortie console et son statut.
CI / CD	Intégration continue : chaque modification est construite et testée automatiquement. Livraison / déploiement continu : chaque version validée est prête à être déployée, ou déployée automatiquement.
Credential	Identifiant stocké de façon chiffrée par Jenkins (mot de passe, clé, GitHub App...), utilisé par les jobs sans être affiché.
Déclencheur (<i>trigger</i>)	Ce qui lance un build : un clic, une scrutation, un webhook, la fin d'un autre job.
GitHub App	Identité « machine » déclarée dans GitHub, avec ses propres permissions, utilisée par Jenkins pour accéder aux dépôts.
Image multi-stage	Dockerfile en plusieurs étapes (FROM ... AS nom) : par exemple une étape de build et de test, puis une image finale légère.
Jenkinsfile	Fichier, versionné avec le code, qui décrit un pipeline.
Job (projet)	Tâche configurée dans Jenkins : free-style (configuré dans l'interface) ou pipeline (décrit par un script).
Pipeline	Job décrit par un script, découpé en stages.
Plugin	Extension de Jenkins (Git, GitHub, Pipeline...). Ceux du TP sont listés dans jenkins/plugins.txt .
Scrutation (<i>polling</i>)	Jenkins interroge régulièrement le dépôt pour détecter les nouveaux commits.
Smoke test	Test rapide, après déploiement, qui vérifie que l'application démarre et répond.
Socket Docker	Fichier /var/run/docker.sock par lequel on pilote le démon Docker. Y avoir accès équivaut à être administrateur de la machine.
Stage	Étape nommée d'un pipeline (Tests, Build...), affichée dans la vue des stages.

Terme	Définition
Webhook	Requête HTTP envoyée par GitHub à une URL (ici Jenkins) lorsqu'un événement se produit (un push).
Workspace	Dossier de travail d'un job, dans lequel Jenkins clone le code : <code>/var/jenkins_home/workspace/<job></code> .

7.3. Index des problèmes fréquents

Message ou symptôme	Voir
Bind for 0.0.0.0:8080 failed: port is already allocated	Étape 1
permission denied while trying to connect to the Docker daemon socket	Étape 1
Couldn't find any revision to build	Étape 3
no configuration file provided: not found	Étape 4
docker-compose: not found	Étape 4
container name "/calculatrice-app" is already in use	Étape 4
curl: (7) Failed to connect to host.docker.internal	Étape 5
target stage "builder" could not be found	Étape 6
Connection timed out (SSH)	Étape 7
Permission denied (publickey)	Étape 7
Webhook : timed out, 302, 404	Étape 8
Deux builds par push	Étapes 8 et 9
Authentication failed / repository not found au clonage	Étape 9